

算法刷题与 C++ 面试教材

数据结构、算法设计、复杂度证明与工程化题解

从暴力枚举到优化推理，从容器原理到面试表达

面向长期能力建设的完整学习稿

前言

这本书的目标不是堆题解，也不是把学习压缩进某个固定周期，而是把数据结构与算法训练成一套可以长期迁移的工程能力。面试题看似零散，背后其实围绕两条主线：一条是数据结构（data structure），也就是信息怎样存、怎样查、怎样更新；另一条是算法（algorithm），也就是在约束下怎样组织计算、怎样减少重复工作、怎样证明结果正确。数据结构和算法不能分开学。只会说“这题用滑动窗口”，但不知道窗口状态应该放在数组、哈希表还是队列里，代码一定会不稳；只会背 `unordered_map`，但不知道它解决的是历史查询问题，也很难在新题里想到它。

本书按“知识完整性”组织，而不是按倒计时组织。前面的章节先建立复杂度、暴力法、复盘方法和 C++ 代码标准；中间把数组、字符串、链表、栈、队列、哈希表、堆、树、图和并查集讲成数据结构主线；后面再进入双指针、滑动窗口、前缀和、二分、搜索、动态规划、回溯、贪心和综合题。训练周期可以按短期集中训练、学期式推进或长期滚动复习来调整，但正文必须把原理、背景、案例、代码、证明、边界和复盘方法讲完整。

每一类算法都必须从暴力方法开始。暴力方法不是低级答案，而是理解问题空间的方式。两数之和先枚举下标对，才能看出慢点是反复查找补数；最短子数组先枚举所有区间，才能看出正数数组存在窗口单调性；岛屿数量先把每个格子当成节点，才能把题目翻译成图的连通块。优化不是凭感觉，而是来自重复工作、单调性、可维护状态、候选淘汰、最优子结构和数据结构查询能力。

代码使用 C++20。示例代码优先使用 `const` 引用、明确边界、清楚的变量名和可测试函数。书里会解释 `vector`、`string`、`deque`、`stack`、`queue`、`priority_queue`、`unordered_map`、`set`、树、图和并查集的用法与底层复杂度，也会说明迭代器失效、哈希退化、扩容、拷贝、溢出和递归栈这些真实面试和线上评测里会遇到的问题。

读法

读这本书时，不要只看最终代码。每道题至少走四遍：第一遍只读题并写暴力思路；第二遍找慢点和可优化性质；第三遍写 C++ 优化代码；第四遍复盘边界、复杂度和面试表达。

每天训练时，把题目记录成固定格式：题型、输入规模、暴力枚举对象、优化来源、不变量、使用的数据结构、复杂度、错因和一周后是否能独立重写。只写“没想到”没有意义，必须写成具体原因：没有发现单调性、哈希表插入顺序错、窗口收缩条件错、二分边界没有排除 `mid`、`int` 溢出、递归深度过大、容器选择导致复杂度不对。

本书目录只显示部和章，小节不进入目录。章内标题用于串联正文，不把内容切碎成大量编号。你应该把每章当成一条推理链来读，而不是把每个小标题当成孤立知识点。

目录

第一部分	学习路线：从基础到综合能力	
第 1 章	刷题方法：从题目到可复用能力	2
第 2 章	训练路线：从基础题到综合能力	3
第 3 章	复杂度、暴力法和优化来源	4
第二部分	数据结构：算法能力的骨架	
第 4 章	线性数据结构：数组、字符串、链表、栈和队列	6
第 5 章	非线性数据结构：哈希表、堆、树、图和并查集	11
第三部分	核心算法：从暴力到优化	
第 6 章	数组、双指针和滑动窗口	18
第 7 章	哈希表和前缀和	23
第 8 章	二分查找和答案空间	26
第 9 章	树、图、BFS 和 DFS	29
第 10 章	回溯、枚举和搜索空间剪枝	36
第 11 章	动态规划：状态、转移和重复子问题	42
附录 A	力扣刷题手册	46

第零部分

学习路线：从基础到综合能力

第 1 章

刷题方法：从题目到可复用能力

刷题和面试训练的核心不是“见过多少题”，而是能否把陌生题拆成熟悉结构。题目只是外壳，真正要识别的是数据怎么组织、状态怎么变化、答案依赖哪些历史信息、是否存在单调性、是否能淘汰候选、是否有重叠子问题。

面对一道题，第一步不是想模板，而是读约束。输入规模决定复杂度上限，值域决定能不能用计数数组或位图，有序性决定能不能双指针或二分，是否允许重复决定哈希表和去重逻辑，是否有负数决定滑动窗口和前缀和的选择。很多错误题解不是算法错，而是约束没看清。

第二步写暴力方法。暴力方法要明确枚举对象：枚举元素、下标对、子数组、子序列、路径、排列、状态，还是答案值。枚举对象一旦明确，复杂度就能估出来。比如两数之和枚举下标对是 $O(n^2)$ ；所有子数组是 $O(n^2)$ ；所有排列是 $O(n!)$ ；网格 BFS 扫所有格子是 $O(mn)$ 。

第三步找重复工作。重复工作有几种典型形态：反复查询某个历史值，适合哈希表；反复求区间和，适合前缀和；反复维护窗口内统计，适合滑动窗口；反复找当前最大或最小，适合堆或单调队列；反复搜索同一状态，适合记忆化或动态规划。

面试表达原则

不要直接说“这题用哈希表”。更好的表达是：暴力会枚举所有下标对，时间是 $O(n^2)$ ；慢点在于每个数都要反复查找另一个数是否存在；我从左到右扫描，用哈希表保存已经看到的值到下标的映射；处理当前值时查 `target - nums[i]` 是否出现过；这样每个元素只插入和查询一次，平均时间 $O(n)$ ，空间 $O(n)$ 。

第四步写不变量。不变量是优化算法正确性的核心。滑动窗口的不变量可能是“窗口内没有重复字符”；单调栈的不变量可能是“栈内下标对应的值单调递减”；BFS 的不变量是“当前层所有节点距离源点相同”；二分的不变量是“答案一定在当前左右边界之间”。没有不变量，代码只是碰巧通过样例。

第五步落到 C++。C++ 代码要讲容器选择。`vector` 适合连续存储和下标访问；`unordered_map` 适合平均常数查询历史信息；`deque` 适合两端弹入弹出；`priority_queue` 适合动态极值但不能删除任意元素；`set` 适合有序前驱后继；图的邻接表通常用 `vector<vector<int>>`，稀疏图比邻接矩阵更节省空间。

Listing 1.1 两数之和：暴力与哈希优化的共同接口

```
std::vector<int> two_sum_hash_table(const std::vector<int>& nums, int target)
{
    std::unordered_map<int, int> value_to_index;
    value_to_index.reserve(nums.size());

    for (int index = 0; index < static_cast<int>(nums.size()); ++index) {
        const int need = target - nums[index];
        const auto found = value_to_index.find(need);
        if (found != value_to_index.end()) {
            return {found->second, index};
        }
        value_to_index.emplace(nums[index], index);
    }
    return {};
}
```

这段代码有几个细节。参数用 `const std::vector<int>&`，避免复制输入数组。哈希表先查再插入，避免同一个元素和自己配对。`reserve` 不是必须，但能减少扩容和 rehash。返回空数组表示没有答案，真实力扣题如果保证有答案，也可以直接返回。

训练路线：从基础题到综合能力

学习路线不是倒计时。有人一个月集中准备，有人一边工作一边长期训练，也有人先补 C++ 和数据结构基础，再进入高强度题目练习。因此本章只给出能力阶段和训练节奏，不把整本书绑定到固定月份。你可以把每个阶段压缩成一周，也可以展开成一个月；真正不能省略的是每个阶段要掌握的知识、题型、代码能力和复盘方法。

第一阶段建立基础语言和复杂度意识。目标是能读懂输入规模，估计可接受复杂度，写出暴力解，并说清楚暴力解慢在哪里。这个阶段学习数组、字符串、`vector`、`string`、排序、二分库函数、下标边界、引用传参和整数溢出。推荐题包括两数之和、移动零、删除有序数组重复项、合并两个有序数组、最大子数组和、反转字符串和有效回文。做题时不要急着背模板，先把枚举对象写清楚：枚举元素、下标对、区间，还是答案边界。

第二阶段学习线性结构和窗口类算法。双指针、滑动窗口、前缀和、单调栈和单调队列看起来像技巧，本质都是“线性序列上的状态维护”。窗口题要写出左右边界的含义，说明什么时候扩张、什么时候收缩；单调结构题要说明哪些候选被淘汰，为什么淘汰后不会丢答案。推荐题包括盛最多水的容器、三数之和、长度最小的子数组、无重复字符的最长子串、找到字符串中所有字母异位词、最小覆盖子串、每日温度、柱状图最大矩形和滑动窗口最大值。

第三阶段把哈希、堆、树和图串起来。哈希表负责历史查询，堆负责动态极值，树负责层次和有序关系，图负责任意状态转移。这个阶段要形成建模习惯：节点是什么，边是什么，状态保存在哪里，访问标记何时更新，答案在入队、出队、入栈、出栈还是回溯时产生。推荐题包括字母异位词分组、最长连续序列、和为 K 的子数组、前 K 个高频元素、数组中的第 K 个最大元素、二叉树层序遍历、验证二叉搜索树、最近公共祖先、岛屿数量、腐烂的橘子、课程表和课程表 II。

第四阶段进入搜索、回溯和动态规划。回溯题要写清选择列表、路径状态、终止条件和撤销动作；动态规划题要写清状态定义、转移来源、初始化、遍历顺序和空间优化条件。不要先背方程，先问一个状态是否足够描述未来。推荐题包括全排列、子集、组合总和、括号生成、单词搜索、爬楼梯、打家劫舍、零钱兑换、最长递增子序列、最长公共子序列、编辑距离、分割等和子集和目标。

第五阶段处理贪心、并查集和综合题。贪心题的关键是证明局部选择为什么能推出全局最优；并查集题的关键是把动态连通关系建模成集合合并；综合题则常常把哈希、排序、堆、图和 DP 混在一起。推荐题包括跳跃游戏、加油站、合并区间、冗余连接、账户合并、实现 Trie、网络连接、单词接龙和数据流中位数。这个阶段不要只追求 AC，要训练面试表达：暴力解是什么，瓶颈在哪里，优化依赖哪条性质，代码如何保证边界正确。

如果需要集中训练，可以把这些阶段压成十二周：基础复杂度与数组字符串，双指针与窗口，哈希与前缀和，栈队列堆与单调结构，二分查找，链表与树，图和拓扑排序，回溯，一维动态规划，二维动态规划和背包，贪心与并查集，综合模拟与查漏补缺。这只是节奏表，不是本书的边界。书的目标是把知识完整讲清楚，训练周期由你的基础、时间和目标岗位决定。

阶段交付

每个阶段至少留下四类材料：一组代表题的暴力解和优化解，一份数据结构选择说明，一份 C++ 容器和边界错误清单，一份一周后能独立重写的回访题表。刷题报告不是形式，它是把题解转化成长期能力的工具。

第 3 章

复杂度、暴力法和优化来源

复杂度是算法设计的预算。输入规模如果是 $n \leq 100$, $O(n^3)$ 可能可以接受; 如果是 $n \leq 2 \times 10^5$, 通常要接近 $O(n \log n)$ 或 $O(n)$; 如果网格规模是 $m * n \leq 10^5$, BFS 或 DFS 扫一遍通常合理; 如果状态空间是指数级, 就要考虑剪枝、记忆化或动态规划。

暴力法的作用是把问题完整展开。两数之和暴力枚举所有下标对, 最长无重复子串暴力枚举所有起点和终点, 岛屿数量暴力从每个未访问陆地扩展连通块。暴力方法慢, 但它通常是正确性的起点。

Listing 3.1 两数之和暴力解

```
std::vector<int> two_sum_bruteforce(const std::vector<int>& nums, int target)
{
    for (int left = 0; left < static_cast<int>(nums.size()); ++left) {
        for (int right = left + 1; right < static_cast<int>(nums.size()); ++right) {
            if (nums[left] + nums[right] == target) {
                return {left, right};
            }
        }
    }
    return {};
```

这段代码枚举的是下标对, 数量约为 $n * (n - 1) / 2$, 时间复杂度是 $O(n^2)$ 。它慢在每个数都要重新扫描后面的数。优化思路不是凭空出现, 而是把“扫描后面所有数”改成“快速查询某个补数是否出现过”。这就是哈希表。

常见优化来源有六类。第一类是缓存重复计算, 例如前缀和、记忆化搜索和动态规划。第二类是单调性, 例如二分、双指针和某些滑动窗口。第三类是局部状态可维护, 例如字符计数、窗口和、不同元素个数。第四类是候选淘汰, 例如单调栈和单调队列。第五类是数据结构查询能力, 例如哈希表、堆、平衡树、并查集。第六类是改变建模方式, 例如把网格问题建成图, 把区间问题排序, 把字符串问题转为频次。

写完暴力法后, 应该固定问几个问题: 我枚举的对象是什么? 有没有重复计算同一段信息? 是否只需要历史信息的一小部分? 是否存在有序或单调? 是否可以把区间查询变成前缀差? 是否可以用哈希把线性查询变成平均常数? 是否可以把指数搜索合并成状态 DP?

面试时要把这些问题讲出来。一个好答案通常不是从“我用某某模板”开始, 而是从暴力法、瓶颈和优化依据开始。

第零部分

数据结构：算法能力的骨架

第 4 章

线性数据结构：数组、字符串、链表、栈和队列

数据结构不是算法之前的一张 API 表，而是算法能够成立的物理基础。刷题时遇到一个问题，第一步不是急着套模板，而是判断答案依赖什么信息：需要按位置访问、按最近关系回退、按先进先出扩展，还是只在两端维护候选。数组、字符串、链表、栈、队列和双端队列构成了这条主线。它们的抽象接口很简单，但背后的存储方式、缓存局部性、迭代器失效规则和边界条件，决定了暴力方法能不能被优化成面试可接受的复杂度。

线性结构的共同特点是元素被组织成一条序列。区别在于这条序列是否连续存储，是否支持下标随机访问，插入删除发生在什么位置，算法状态如何随着左右边界移动而更新。真正掌握这一章，意味着看到题目时可以回答四个问题：序列是否固定，窗口是否连续，历史状态能否增量维护，删除或回退是否只发生在某一端。

从能力边界看，**vector** 负责连续存储、随机访问和顺序扫描，是 DP 表、排序数组、邻接表和双指针题的默认选择；**string** 负责字符序列和子串窗口，但要警惕 **substr** 的复制成本；链表负责节点重连，适合反转、合并、环检测和快慢指针；**stack** 负责最近未解决状态，常用于括号、表达式、路径和单调栈；**queue** 负责按层扩展，是 BFS 的自然结构；**deque** 负责两端维护候选，尤其适合滑动窗口极值。把这些结构先放在同一张能力地图里，后面每个算法就不会变成孤立模板。

数组和 **std::vector** 是刷题默认容器。连续存储带来两个关键优势：下标访问是 $O(1)$ ，相邻元素通常落在相近内存位置，CPU 缓存命中率高。很多初学者只记住第一个优势，忽略第二个优势。实际程序中，同样是线性扫描，**vector** 往往比链表快得多，并不是因为复杂度写法不同，而是因为连续内存更容易被硬件预取。代价也来自连续存储：中间插入和删除必须移动后续元素；容量不足时扩容会重新申请一段更大的空间，把旧元素移动或复制过去，原来的指针、引用和迭代器都可能失效。

Listing 4.1 vector 的容量、大小和失效规则

```
std::vector<int> nums;
nums.reserve(n);      // 只预留容量，不构造元素，size 仍然是 0。
nums.push_back(10);
nums.emplace_back(20);

std::vector<int> dp(n + 1, 0); // 直接创建 n + 1 个元素。

const int* first = nums.data();
nums.push_back(30);          // 如果触发扩容，first 立刻失效。
```

reserve 和 **resize** 必须分清。**reserve** 只改变容量，不构造元素，适合已经知道将要追加多少元素的场景；**resize** 改变大小，会构造或销毁元素，适合直接创建 DP 数组、计数数组或矩阵行。写 **vector<int> dp(n + 1)** 后，**dp[i]** 是合法的；只写 **reserve(n + 1)** 后访问 **dp[i]** 是越界。面试里这类错误很隐蔽，因为代码可能在小样例上运行，到了评测环境才崩溃。

数组题的暴力方法通常是枚举一个区间、一个配对或一个分割点。优化的关键，是让“下一步答案”复用“上一步状态”。例如区间和暴力做法是每次重新累加 **nums[left..right]**，复杂度 $O(n^3)$ 或 $O(n^2)$ ；前缀和把任意区间和转成两个端点差值，使查询变成 $O(1)$ 。滑动窗口则在区间右端加入一个元素、左端删除一个元素，用增量更新维护窗口状态。它们都依赖数组按下标稳定访问。

Listing 4.2 从重新求和到前缀和查询

```
std::vector<long long> build_prefix_sum(const std::vector<int>& nums)
{
    std::vector<long long> prefix(nums.size() + 1, 0);
    for (std::size_t i = 0; i < nums.size(); ++i) {
        prefix[i + 1] = prefix[i] + nums[i];
    }
    return prefix;
}
```

```

long long range_sum(const std::vector<long long>& prefix,
                    std::size_t left,
                    std::size_t right)
{
    return prefix[right + 1] - prefix[left];
}

```

这段代码里 `prefix[i]` 表示前 `i` 个元素之和，因此区间 `[left, right]` 的和是 `prefix[right + 1] - prefix[left]`。多出来的第 0 位看似绕，但它消除了 `left == 0` 的特殊判断。很多高质量题解都会主动设计这种哨兵位，因为它能让边界条件服从同一条公式。

字符串 `std::string` 可以看成字符数组，但不能只按字符数组理解。第一，`substr` 会创建新字符串，频繁调用会引入大量复制；第二，`size()` 返回字节数，不等于所有自然语言里的“字符数”；第三，题目如果限定小写英文，计数结构可以用 `std::array<int, 26>`，如果字符集不固定，就应使用哈希表。刷题常见输入大多是 ASCII 或小写英文，面试时仍然要主动说明自己的假设。

Listing 4.3 异位词窗口：避免反复 `substr`

```

std::vector<int> find_anagrams(const std::string& s, const std::string& p)
{
    constexpr int ALPHABET_SIZE = 26;
    std::vector<int> answer;
    if (s.size() < p.size()) {
        return answer;
    }

    std::array<int, ALPHABET_SIZE> need{};
    std::array<int, ALPHABET_SIZE> window{};
    for (const char ch : p) {
        ++need[ch - 'a'];
    }

    for (std::size_t right = 0; right < s.size(); ++right) {
        ++window[s[right] - 'a'];
        if (right >= p.size()) {
            --window[s[right - p.size()] - 'a'];
        }
        if (window == need) {
            answer.push_back(static_cast<int>(right + 1 - p.size()));
        }
    }
    return answer;
}

```

暴力版本会枚举每个长度为 `p.size()` 的子串，复制出来再排序或统计。复制子串本身就是 $O(k)$ ，排序还要 $O(k \log k)$ 。窗口版本只维护 26 个计数，右端进入一个字符，左端离开一个字符，每步常数更新。优化成立的原因不是“用了滑动窗口”这个名词，而是题目只关心固定长度窗口内的字符频次，窗口相邻状态只差两个字符。

链表是线性结构里最容易被误解的一个。教材常说链表插入删除是 $O(1)$ ，但这句话有前提：你已经拿到了要操作位置的前驱节点或当前节点。如果还需要从头查找位置，查找本身就是 $O(n)$ 。链表的价值不在于比数组更快，而在于节点位置稳定、局部重连便宜、结构可以自然表达“下一个”。面试链表题考的不是容器 API，而是指针不丢、边界不漏、连接顺序清楚。

Listing 4.4 迭代反转链表：先保存后继，再改变指向

```

struct ListNode {
    int value = 0;
    ListNode* next = nullptr;
};

ListNode* reverse_list(ListNode* head)
{
    ListNode* previous = nullptr;
    ListNode* current = head;

    while (current != nullptr) {
        ListNode* next = current->next;

```

```

        current->next = previous;
        previous = current;
        current = next;
    }

    return previous;
}

```

这段代码只有四行核心操作，顺序却不能乱。先保存 `next`，因为一旦执行 `current->next = previous`，原来的后继就找不到了；再把当前节点接到已反转部分前面；最后整体向后推进。写链表题时，建议在纸上画三个指针：已经处理好的部分、当前节点、尚未处理的部分。只要这三段关系清楚，反转、两两交换、K 个一组反转都可以从同一套指针动作推出来。

虚拟头节点是链表题的另一个重要技巧。删除倒数第 N 个节点、合并两个链表、分隔链表时，真实头节点可能被删除或替换。如果每次都单独判断头节点，代码会被边界分支切碎。虚拟头节点把“修改头节点”和“修改普通节点”统一成“修改某个前驱节点的 `next`”。

Listing 4.5 虚拟头节点统一删除逻辑

```

ListNode* remove_nth_from_end(ListNode* head, int n)
{
    ListNode dummy;
    dummy.next = head;

    ListNode* fast = &dummy;
    ListNode* slow = &dummy;

    for (int step = 0; step < n; ++step) {
        fast = fast->next;
    }
    while (fast->next != nullptr) {
        fast = fast->next;
        slow = slow->next;
    }

    ListNode* removed = slow->next;
    slow->next = removed->next;
    return dummy.next;
}

```

快慢指针的本质是让两个指针之间保持固定距离或固定速度差。删除倒数第 N 个节点时，快指针先走 N 步，之后快慢一起走；当快指针到尾部，慢指针正好停在待删节点前驱。环检测时，快指针每次走两步，慢指针每次走一步；如果存在环，速度差会让它们在环内相遇。不要把快慢指针背成模板，应该说清楚“两个指针之间的不变量是什么”。

栈是后进先出结构，适合处理“最近出现、最先解决”的关系。括号匹配中，最近的左括号必须先被右括号匹配；路径简化中，最近进入的目录会被 `..` 回退；表达式求值中，最近的运算符和操作数构成局部归约。栈的强大之处在于把嵌套结构压平成一条处理过程。

Listing 4.6 有效括号：最近未匹配结构

```

bool is_valid_parentheses(const std::string& s)
{
    std::vector<char> stack;
    stack.reserve(s.size());

    for (const char ch : s) {
        if (ch == '(' || ch == '[' || ch == '{') {
            stack.push_back(ch);
            continue;
        }
        if (stack.empty()) {
            return false;
        }

        const char left = stack.back();
        stack.pop_back();
        const bool matched = (left == '(' && ch == ')') ||
                               (left == '[' && ch == ']') ||

```

```

        (left == '{' && ch == '}');

        if (!matched) {
            return false;
        }
    }

    return stack.empty();
}

```

这里直接使用 `std::vector<char>` 作为栈，比 `std::stack<char>` 更方便查看和预留容量。`std::stack` 是容器适配器，只暴露 `push`、`pop`、`top`、`empty` 等接口；`pop()` 不返回元素，因此必须先 `top()` 再 `pop()`。面试写代码时两者都可以，但要明白适配器牺牲了一部分访问能力，换来更明确的抽象边界。

单调栈是栈在刷题中的高频变体。它维护的不是时间顺序本身，而是一组“尚未找到答案的候选”。每日温度中，栈保存还没有找到更高温度的下标，并且这些下标对应的温度从栈底到栈顶单调不减。新温度到来时，只要它高于栈顶下标的温度，就可以确定栈顶答案；因为这是栈顶下标右侧第一个更高温度的下标。

Listing 4.7 每日温度：单调栈保存下标

```

std::vector<int> daily_temperatures(const std::vector<int>& temperatures)
{
    std::vector<int> answer(temperatures.size(), 0);
    std::vector<int> stack;
    stack.reserve(temperatures.size());

    for (int day = 0; day < static_cast<int>(temperatures.size()); ++day) {
        while (!stack.empty() && temperatures[day] > temperatures[stack.back()]) {
            const int previous_day = stack.back();
            stack.pop_back();
            answer[previous_day] = day - previous_day;
        }
        stack.push_back(day);
    }

    return answer;
}

```

单调栈常保存下标而不是值，因为题目经常需要距离、宽度，或者需要回到原数组访问更多信息。它能把暴力枚举“右侧第一个更大元素”的 $O(n^2)$ 降到 $O(n)$ ，原因是每个下标最多入栈一次、出栈一次。这个摊还分析会说：虽然某一次循环里可能弹出很多元素，但全局弹出次数不超过 n 。

队列是先进先出结构，适合按层扩展。BFS 的正确性来自队列顺序：先入队的是距离更短或时间更早的状态，因此第一次到达一个未访问状态时，就已经得到了最短步数。二叉树层序遍历、腐烂橘子、多源最短扩散、无权图最短路，都依赖这个性质。与栈不同，队列不会深挖某一条路径，而是把同一层的状态先处理完。

Listing 4.8 层序遍历：队列保存当前边界

```

std::vector<std::vector<int>> level_order(TreeNode* root)
{
    std::vector<std::vector<int>> levels;
    if (root == nullptr) {
        return levels;
    }

    std::queue<TreeNode*> queue;
    queue.push(root);

    while (!queue.empty()) {
        const int level_size = static_cast<int>(queue.size());
        std::vector<int> level;
        level.reserve(level_size);

        for (int i = 0; i < level_size; ++i) {
            TreeNode* node = queue.front();
            queue.pop();
            level.push_back(node->value);

            if (node->left) queue.push(node->left);
            if (node->right) queue.push(node->right);
        }

        levels.push_back(level);
    }

    return levels;
}

```

```

        if (node->left != nullptr) {
            queue.push(node->left);
        }
        if (node->right != nullptr) {
            queue.push(node->right);
        }
    }
    levels.push_back(std::move(level));
}

return levels;
}

```

这段代码里的 `level_size` 是关键。没有它，也可以 BFS，但很难知道哪些节点属于同一层。层序遍历、最短时间、多源扩散都需要把“当前层的节点数量”固定下来，然后处理完这一层再增加距离或分钟数。

`std::deque` 支持两端 $O(1)$ 插入删除，是 `std::queue` 的默认底层容器，也是单调队列的常用实现。滑动窗口最大值的暴力方法是每个窗口扫描一遍最大值，复杂度 $O(nk)$ 。如果用堆，需要处理过期下标。单调队列更贴合题意：队头永远是当前窗口最大值下标，队尾负责淘汰不可能再成为最大值的候选。

Listing 4.9 滑动窗口最大值：单调队列

```

std::vector<int> max_sliding_window(const std::vector<int>& nums, int k)
{
    std::vector<int> answer;
    if (nums.empty() || k <= 0) {
        return answer;
    }
    answer.reserve(nums.size());

    std::deque<int> deque;
    for (int right = 0; right < static_cast<int>(nums.size()); ++right) {
        while (!deque.empty() && nums[deque.back()] <= nums[right]) {
            deque.pop_back();
        }
        deque.push_back(right);

        const int left = right - k + 1;
        if (deque.front() < left) {
            deque.pop_front();
        }
        if (left >= 0) {
            answer.push_back(nums[deque.front()]);
        }
    }

    return answer;
}

```

单调队列的优化理由和单调栈相似。一个新元素如果大于等于队尾元素，那么队尾元素在未来任何包含新元素的窗口里都不可能成为最大值，因为它更小且更早过期。于是可以安全删除。每个下标最多进队一次、出队一次，总复杂度 $O(n)$ 。

线性结构选型

需要稳定下标和顺序扫描，优先 **vector**；需要字符窗口，使用 **string** 加计数结构，不要频繁 **substr**；需要局部重连，考虑链表并画清楚前驱、当前、后继；需要最近未解决状态，用栈；需要按层或最短步数扩展，用队列；需要窗口两端同时维护候选，用双端队列。选择结构时同时说出它让哪一步查询、删除或更新变快。

本章建议配套练习：数组与前缀和做 303、304、560；字符串窗口做 3、438、567、76；链表做 19、21、24、141、142、206、92、25；栈做 20、71、150、155、739、84；队列和双端队列做 102、199、239、994。复盘时不要只写最终代码，要写三行说明：暴力方法慢在哪里，数据结构保存了什么状态，为什么每个元素只被处理有限次。

第 5 章

非线性数据结构：哈希表、堆、树、图和并查集

线性结构擅长处理顺序，非线性结构擅长处理关系和查询。哈希表回答“这个状态以前出现过吗”；堆回答“当前最极端的候选是谁”；树表达层次、有序性和递归子问题；图表达任意状态之间的连接；并查集维护“这些元素现在是否属于同一组”。这些结构不是刷题技巧的附属品，而是算法优化的来源。很多题从 $O(n^2)$ 变成 $O(n)$ ，并不是因为循环少写了一层，而是因为某个昂贵查询被换成了合适的数据结构查询。

哈希表是保存历史信息的核心结构。`std::unordered_map` 和 `std::unordered_set` 的平均查询、插入、删除复杂度是 $O(1)$ ，适合把“回头查找”变成“即时查询”。两数之和的暴力方法枚举两个下标，慢在每个数都要向后找补数；哈希表版本边扫描边记录已经见过的值，当前数只需要查询一次补数。这个优化成立的前提是题目只要求找到一对下标，且补数是否出现过可以由历史集合回答。

Listing 5.1 两数之和：哈希表把补数查询变成平均常数时间

```
std::vector<int> two_sum(const std::vector<int>& nums, int target)
{
    std::unordered_map<int, int> index_by_value;
    index_by_value.reserve(nums.size());

    for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
        const int complement = target - nums[i];
        const auto found = index_by_value.find(complement);
        if (found != index_by_value.end()) {
            return {found->second, i};
        }
        index_by_value.emplace(nums[i], i);
    }

    return {};
}
```

代码里先查再插入，是为了避免同一个元素被用两次。例如目标是 6，当前数是 3，如果先插入再查，可能错误地返回同一个下标。这里用 `find` 而不是 `operator[]`，因为 `operator[]` 在 key 不存在时会插入默认值。只想判断存在性时，C++20 可以用 `contains`；需要同时取值时，用 `find`，这样只查一次。刷题中还应该习惯 `reserve`，它能减少 rehash 次数。rehash 会重新组织桶，导致迭代器失效；如果保存了哈希表元素迭代器，扩容后不能继续使用。

哈希表不仅能存值到下标，也能存频次、前缀和、状态编码和访问标记。前缀和加哈希表是很重要的一类优化。和为 K 的子数组如果暴力枚举左右端点，需要 $O(n^2)$ ；如果知道当前位置前缀和是 `current`，那么以当前位置结尾、和为 k 的子数组数量，等于以前出现过多少次 `current - k`。哈希表保存的是“某个前缀和出现次数”，查询的是“能和当前前缀配成目标的历史前缀”。

Listing 5.2 和为 K 的子数组：前缀和频次

```
int subarray_sum_equals_k(const std::vector<int>& nums, int k)
{
    std::unordered_map<int, int> prefix_count;
    prefix_count.reserve(nums.size() + 1);
    prefix_count.emplace(0, 1);

    int prefix = 0;
    int answer = 0;
    for (const int x : nums) {
        prefix += x;
        const auto found = prefix_count.find(prefix - k);
        if (found != prefix_count.end()) {
            answer += found->second;
        }
    }
}
```

```

        ++prefix_count[prefix];
    }

    return answer;
}

```

这个算法容易错在初始化。空前缀和 0 必须先出现一次，否则从下标 0 开始的合法子数组会漏掉。它也解释了为什么有负数时不能随便用滑动窗口：窗口和不再随右端单调增加，左端移动也不保证更接近目标；前缀和加哈希表不依赖正数单调性，所以适用范围更广。

哈希表的限制同样要清楚。第一，它不维护顺序，如果题目要求按排序顺序取最小、最大或区间，就该考虑 `std::map`、`std::set`、堆、树状数组或线段树。第二，平均 $O(1)$ 不等于绝对 $O(1)$ ，糟糕哈希或恶意数据可能退化。第三，自定义 key 需要提供相等比较和哈希函数。面试里一般不会刻意卡哈希攻击，但你要知道 `unordered_map<pair<int,int>, int>` 不能直接在所有标准库环境下工作，需要自定义哈希。

Listing 5.3 为 pair 坐标提供哈希函数

```

struct PairHash {
    std::size_t operator()(const std::pair<int, int>& point) const noexcept
    {
        constexpr unsigned COORDINATE_BITS = 32U;
        const auto first = static_cast<std::uint64_t>(
            static_cast<std::uint32_t>(point.first));
        const auto second = static_cast<std::uint64_t>(
            static_cast<std::uint32_t>(point.second));
        return std::hash<std::uint64_t>{}((first << COORDINATE_BITS) ^ second);
    }
};

std::unordered_set<std::pair<int, int>, PairHash> visited;

```

堆解决的是动态极值问题。C++ 里通常使用 `std::priority_queue`，默认是大顶堆；如果要小顶堆，需要显式给出容器和比较器。堆的插入和弹出堆顶都是 $O(\log n)$ ，查看堆顶是 $O(1)$ 。它不适合频繁删除任意元素，也不适合遍历出有序结果而不破坏结构。换句话说，堆只承诺“我能很快告诉你当前最大或最小是谁”。

Listing 5.4 `priority_queue` 的大顶堆和小顶堆

```

std::priority_queue<int> max_heap;

std::priority_queue<int,
    std::vector<int>,
    std::greater<int>> min_heap;

max_heap.push(3);
max_heap.push(10);
const int largest = max_heap.top();
max_heap.pop();

```

Top K 是堆最经典的用法。求数组中第 K 大元素，直觉做法是排序，复杂度 $O(n \log n)$ 。如果只关心前 K 大，不需要完整排序，可以维护一个大小为 K 的小顶堆。堆顶是当前前 K 大中最小的元素；新元素如果不超过堆顶，就不可能进入前 K；如果超过堆顶，就弹出堆顶并插入新元素。最终堆顶就是第 K 大。

Listing 5.5 第 K 大元素：大小为 K 的小顶堆

```

int find_kth_largest(const std::vector<int>& nums, int k)
{
    std::priority_queue<int,
        std::vector<int>,
        std::greater<int>> heap;

    for (const int x : nums) {
        if (static_cast<int>(heap.size()) < k) {
            heap.push(x);
        } else if (x > heap.top()) {

```



```

        heap.pop();
        heap.push(x);
    }
}

return heap.top();
}

```

这个方法的复杂度是 $O(n \log k)$ ，适合 k 远小于 n 的场景。还有一种平均线性复杂度的快速选择算法，后文会讲。面试表达时要说明选择堆的理由：不需要完整有序，只需要维护 K 个候选的边界。类似地，合并 K 个升序链表用小顶堆保存每条链表当前头节点；任务调度用堆保存当前最早结束或最高优先级任务；数据流中位数用一个大顶堆和一个小顶堆分别维护较小一半和较大一半。

树把数据组织成层次关系。二叉树题看似在考递归，实际在考状态如何沿边传播。状态可以从父节点传给子节点，例如验证二叉搜索树时传递取值上下界；也可以从子节点汇总到父节点，例如求树的高度、直径、最大路径和；还可以按层从左到右传播，例如层序遍历。只背前序、中序、后序三个词是不够的，必须知道遍历顺序服务于哪类状态。

Listing 5.6 二叉搜索树验证：显式栈中序遍历

```

bool is_valid_bst(TreeNode* root)
{
    std::vector<TreeNode*> stack;
    TreeNode* current = root;
    std::optional<long long> previous;

    while (current != nullptr || !stack.empty()) {
        while (current != nullptr) {
            stack.push_back(current);
            current = current->left;
        }

        current = stack.back();
        stack.pop_back();

        if (previous.has_value() && current->value <= previous.value()) {
            return false;
        }
        previous = current->value;
        current = current->right;
    }

    return true;
}

```

二叉搜索树的关键性质是中序遍历严格递增。验证 BST 的暴力想法可能是对每个节点扫描整棵左子树和右子树，判断左边都小、右边都大，这会重复访问大量节点。中序遍历把全局约束转成相邻访问值的递增关系，每个节点只处理一次。这里用显式栈而不是递归，是为了避免极端深树导致 C++ 调用栈过深，同时也让“遍历状态”更清楚。

树题经常需要区分“节点为空的返回值”和“答案的初始值”。求最大深度时，空节点深度是 0；求最大路径和时，答案不能初始化为 0，因为所有节点可能都是负数；求最近公共祖先时，返回值表示“当前子树是否找到了目标节点或祖先”。这些不是小细节，而是树形 DP 的语义。每个返回值都应该能用一句话解释：它代表当前子树对父节点贡献什么信息。

图是最通用的关系结构。只要有状态和一步转移，就可以建模成图。图题最重要的不是 DFS 或 BFS 写法，而是建模四问：节点是什么，边是什么，访问状态是什么，队列或栈里保存什么。岛屿数量里，节点是陆地格子，边是上下左右相邻；课程表里，节点是课程，有向边表示先修依赖；单词接龙里，节点是单词，边表示一次字符变化；开锁问题里，节点是四位密码状态，边是转动一位数字。

图的存储方式要跟输入规模匹配。稠密图可以用邻接矩阵，判断两点是否有边是 $O(1)$ ，但空间是 $O(n^2)$ 。大多数面试题是稀疏图，更适合邻接表，空间是 $O(n + m)$ 。如果节点编号连续，用 `vector<vector<int>>`；如果节点是字符串或稀疏编号，用哈希表映射到编号，或者直接用 `unordered_map<string, vector<string>>`。

Listing 5.7 课程表：邻接表和入度数组

```

bool can_finish(int num_courses,
                const std::vector<std::vector<int>>& prerequisites)
{
    std::vector<std::vector<int>> graph(num_courses);
    std::vector<int> indegree(num_courses, 0);

    for (const auto& edge : prerequisites) {
        const int course = edge[0];
        const int prerequisite = edge[1];
        graph[prerequisite].push_back(course);
        ++indegree[course];
    }

    std::queue<int> queue;
    for (int course = 0; course < num_courses; ++course) {
        if (indegree[course] == 0) {
            queue.push(course);
        }
    }

    int learned = 0;
    while (!queue.empty()) {
        const int course = queue.front();
        queue.pop();
        ++learned;

        for (const int next : graph[course]) {
            --indegree[next];
            if (indegree[next] == 0) {
                queue.push(next);
            }
        }
    }

    return learned == num_courses;
}

```

拓扑排序的暴力直觉是反复扫描所有课程，找当前没有前置依赖的课。这样每学一门课都可能重新扫描所有边。入度数组把“还有多少依赖没被满足”变成可增量更新的状态；队列保存当前入度为 0 的课程。每条边只在它的起点被移除时处理一次，复杂度 $O(n + m)$ 。如果最后学完的课程数少于总数，说明剩余节点互相依赖，图中有环。

DFS 和 BFS 都可以遍历图，但语义不同。BFS 适合无权最短步数和层次扩散，因为它按距离递增访问状态；DFS 适合连通块标记、路径搜索、拓扑状态检测和回溯枚举，因为它沿一条路径深入。C++ 里递归 DFS 简洁，但深度过大可能栈溢出。本书讲递归思想，但实现上会经常给出显式栈版本，让遍历过程、访问标记和状态回退更加可控。

Listing 5.8 岛屿数量：显式栈标记连通块

```

int num_islands(std::vector<std::vector<char>>& grid)
{
    if (grid.empty() || grid[0].empty()) {
        return 0;
    }

    constexpr char LAND = '1';
    constexpr char WATER = '0';
    constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
        std::pair{1, 0}, std::pair{-1, 0},
        std::pair{0, 1}, std::pair{0, -1}
    };

    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());
    int islands = 0;

    for (int row = 0; row < rows; ++row) {
        for (int col = 0; col < cols; ++col) {
            if (grid[row][col] != LAND) {
                continue;
            }

```

```

++islands;
std::vector<std::pair<int, int>> stack{{row, col}};
grid[row][col] = WATER;

while (!stack.empty()) {
    const auto [r, c] = stack.back();
    stack.pop_back();

    for (const auto [dr, dc] : DIRECTIONS) {
        const int nr = r + dr;
        const int nc = c + dc;
        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols ||
            grid[nr][nc] != LAND) {
            continue;
        }
        grid[nr][nc] = WATER;
        stack.emplace_back(nr, nc);
    }
}

return islands;
}

```

这段代码把访问过的陆地直接改成水，省掉单独的 **visited** 数组。这样做是否合适取决于题目是否允许修改输入。如果不允许，就创建同样大小的布尔矩阵。图题里访问标记必须在入队或入栈时就设置，而不是弹出时才设置；否则同一个节点可能被多个邻居重复加入，轻则浪费时间，重则导致状态爆炸。

并查集维护动态连通性。它有两个核心操作：**find** 找到一个元素所在集合的代表，**unite** 合并两个集合。路径压缩让查找时顺手把沿途节点直接挂到根上，按大小或按秩合并让树保持较浅。两者结合后，单次操作的均摊复杂度接近常数。它适合回答“是否在同一组”“合并后还有几个连通块”，不适合回答两点之间的具体路径或最短距离。

Listing 5.9 并查集：路径压缩与按大小合并

```

class DisjointSetUnion {
public:
    explicit DisjointSetUnion(int n)
        : parent_(n), size_(n, 1)
    {
        std::iota(this->parent_.begin(), this->parent_.end(), 0);
    }

    int find(int x)
    {
        int root = x;
        while (this->parent_[root] != root) {
            root = this->parent_[root];
        }
        while (this->parent_[x] != x) {
            const int next = this->parent_[x];
            this->parent_[x] = root;
            x = next;
        }
        return root;
    }

    bool unite(int a, int b)
    {
        int root_a = find(a);
        int root_b = find(b);
        if (root_a == root_b) {
            return false;
        }
        if (this->size_[root_a] < this->size_[root_b]) {
            std::swap(root_a, root_b);
        }
        this->parent_[root_b] = root_a;
    }
};

```

```

        this->size[root_a] += this->size[root_b];
        return true;
    }

private:
    std::vector<int> parent_;
    std::vector<int> size_;
};

```

冗余连接是并查集的标准例题。给一棵树额外加一条边，依次扫描边。如果一条边连接的两个点已经在同一集合里，那么再加入它就会形成环，这条边就是冗余边。暴力方法可以每次加边后做一次 DFS 检查连通性，复杂度很高；并查集把“两个点是否已经连通”维护成近乎常数时间查询。

账户合并也能体现并查集的建模能力。邮箱是连接账户的证据：两个账户只要共享一个邮箱，就属于同一个人。可以把账户编号作为并查集元素，用哈希表记录邮箱第一次出现在哪个账户；再次看到同一邮箱时，把两个账户合并。最后按根节点收集邮箱并排序。这里哈希表负责从字符串邮箱找到账户编号，并查集负责维护账户之间的连通分量。一个题常常需要多个数据结构协作，这也是“数据结构和算法不分家”的真实含义。

数据结构和算法不分家

优化通常来自查询模型的改变。哈希表把历史查找变快，堆把动态极值变快，树把层次状态和有序性质显式化，图把状态转移统一成边，并查集把动态连通性维护变快。刷题复盘时必须同时写出算法动作和数据结构职责：保存什么，支持什么查询，删除什么过期状态，为什么不会漏答案。

本章建议配套练习：哈希表做 1、49、128、560、146；堆做 215、347、23、295、621；树做 94、98、102、104、105、124、236、230；图做 200、207、210、417、994、127、752；并查集做 547、684、721、990、1319。每道题至少复盘一次建模：如果不用这个结构，暴力瓶颈在哪里；用了这个结构，哪一步查询或合并被降下来了；结构本身有什么边界和失效条件。

第零部分

核心算法：从暴力到优化

第 6 章

数组、双指针和滑动窗口

数组题是刷题的地基。双指针和滑动窗口不是孤立技巧，而是从暴力枚举子结构中优化出来的方法。双指针常用于有序数组、两端收缩和左右边界协作；滑动窗口常用于连续子数组或子串，并且窗口状态能够增量维护。

盛最多水的容器暴力解枚举左右边界。面积由宽度和较短边决定，所以每个下标对都可能成为候选。

Listing 6.1 盛水容器暴力解

```
int max_area_bruteforce(const std::vector<int>& height)
{
    int best = 0;
    for (int left = 0; left < static_cast<int>(height.size()); ++left) {
        for (int right = left + 1; right < static_cast<int>(height.size()); ++right) {
            const int width = right - left;
            const int bounded_height = std::min(height[left], height[right]);
            best = std::max(best, width * bounded_height);
        }
    }
    return best;
}
```

暴力法是 $O(n^2)$ 。优化来自候选淘汰。初始取最宽的左右边界，如果移动较高的一边，宽度变小，而较短边没有变高，面积不可能变大。因此只能移动较短边，才有机会提高瓶颈高度。

Listing 6.2 盛水容器双指针解

```
int max_area_two_pointers(const std::vector<int>& height)
{
    int left = 0;
    int right = static_cast<int>(height.size()) - 1;
    int best = 0;

    while (left < right) {
        const int width = right - left;
        const int bounded_height = std::min(height[left], height[right]);
        best = std::max(best, width * bounded_height);

        if (height[left] <= height[right]) {
            ++left;
        } else {
            --right;
        }
    }
    return best;
}
```

不变量是：每一步排除的短边，不会和更内侧的任何边形成更优答案，因为宽度只会变小，短边高度仍然是瓶颈。这个证明比“左右指针往中间走”重要得多。

长度最小的子数组展示滑动窗口。暴力法枚举所有起点和终点，遇到和达到目标时更新答案。它慢在每个起点都重新累加。

Listing 6.3 长度最小子数组滑动窗口

```
int min_subarray_len_sliding_window(int target, const std::vector<int>& nums)
{
    int best = std::numeric_limits<int>::max();
    int left = 0;
    int sum = 0;

    for (int right = 0; right < static_cast<int>(nums.size()); ++right) {
```

```

        sum += nums[right];
        while (sum >= target) {
            best = std::min(best, right - left + 1);
            sum -= nums[left];
            ++left;
        }
    }

    return best == std::numeric_limits<int>::max() ? 0 : best;
}

```

这个优化依赖正整数。右边界右移时窗口和只会增加，左边界右移时窗口和只会减少。如果数组包含负数，这个单调性消失，滑动窗口可能失效，需要考虑前缀和、单调队列或其他结构。

无重复字符的最长子串也是窗口题，但窗口状态不是和，而是字符最后出现位置。窗口不变量是 `[left, right]` 内没有重复字符。

Listing 6.4 无重复字符最长子串

```

int longest_substring_without_repeat_window(const std::string& s)
{
    constexpr int ASCII_RANGE = 256;
    std::array<int, ASCII_RANGE> last_seen{};
    last_seen.fill(-1);

    int left = 0;
    int best = 0;

    for (int right = 0; right < static_cast<int>(s.size()); ++right) {
        const auto ch = static_cast<unsigned char>(s[right]);
        if (last_seen[ch] >= left) {
            left = last_seen[ch] + 1;
        }
        last_seen[ch] = right;
        best = std::max(best, right - left + 1);
    }

    return best;
}

```

这里用 `std::array<int, 256>` 是因为按 ASCII 字符处理。如果题目要求完整 Unicode 字符，需要先明确字符单元，或使用哈希表保存字符到位置。

三数之和是双指针的第二个标准模型。题目要求找出所有不重复三元组，使得三个数之和为 0。暴力方法枚举三个下标，时间复杂度 $O(n^3)$ ，还要处理去重；如果用哈希表固定两数再找第三数，可以降到 $O(n^2)$ ，但去重和重复值处理仍然复杂。真正稳定的做法是先排序，再固定第一个数，把剩下两个数变成有序数组上的两数之和。

排序后的关键性质是：固定 `first` 后，如果 `nums[left] + nums[right]` 太小，说明左端值不够大，只能右移 `left`；如果和太大，说明右端值太大，只能左移 `right`。这就是有序性带来的候选淘汰。去重也依赖排序：同一层循环里，如果当前值和前一个值相同，就跳过；找到一个答案后，左右指针都要跳过连续重复值。

Listing 6.5 三数之和：排序加双指针

```

std::vector<std::vector<int>> three_sum(std::vector<int> nums)
{
    std::sort(nums.begin(), nums.end());
    std::vector<std::vector<int>> answer;

    for (int first = 0; first < static_cast<int>(nums.size()); ++first) {
        if (first > 0 && nums[first] == nums[first - 1]) {
            continue;
        }
        if (nums[first] > 0) {
            break;
        }

        int left = first + 1;
        int right = static_cast<int>(nums.size()) - 1;
    }
}

```



```

while (left < right) {
    const int sum = nums[first] + nums[left] + nums[right];
    if (sum == 0) {
        answer.push_back({nums[first], nums[left], nums[right]});
        const int left_value = nums[left];
        const int right_value = nums[right];
        while (left < right && nums[left] == left_value) {
            ++left;
        }
        while (left < right && nums[right] == right_value) {
            --right;
        }
    } else if (sum < 0) {
        ++left;
    } else {
        --right;
    }
}

return answer;
}

```

这里参数按值传入,是因为函数内部要排序。如果不想修改调用者的数组,按值传参是合理的;如果题目允许原地修改,也可以传引用。复杂度是排序 $O(n \log n)$ 加双指针枚举 $O(n^2)$, 总时间 $O(n^2)$, 额外空间不计答案为 $O(1)$ 或排序栈空间。面试表达要强调: 去重不是最后把答案放进 set, 而是在生成答案时就避免重复, 这样更可控。

找到字符串中所有字母异位词是固定长度窗口。题目给定字符串 *s* 和模式串 *p*, 要求找出 *s* 中所有长度等于 *p* 且字符频次完全相同的起点。暴力方法枚举每个长度为 *p.size()* 的子串, 然后排序比较或统计比较。如果每次排序, 复杂度是 $O(nk \log k)$; 如果每次重新统计, 复杂度是 $O(nk)$ 。慢点在于相邻窗口高度重叠, 却重复计算了大部分字符频次。

优化方法是维护固定长度窗口的频次数组。窗口右端每进入一个字符, 就把它的计数加一; 当窗口长度超过模式串长度, 左端字符离开, 计数减一。由于题目通常限定小写英文, 可以用 `std::array<int, 26>`, 比较两个数组是常数时间。窗口不变量是: 处理到 *right* 时, *window* 正好记录当前长度不超过模式串长度的后缀窗口字符频次。

Listing 6.6 找到所有字母异位词: 固定长度窗口

```

std::vector<int> find_anagrams_window(const std::string& s,
                                     const std::string& p)
{
    constexpr int ALPHABET_SIZE = 26;
    std::vector<int> answer;
    if (s.size() < p.size()) {
        return answer;
    }

    std::array<int, ALPHABET_SIZE> need{};
    std::array<int, ALPHABET_SIZE> window{};
    for (const char ch : p) {
        ++need[ch - 'a'];
    }

    for (std::size_t right = 0; right < s.size(); ++right) {
        ++window[s[right] - 'a'];
        if (right >= p.size()) {
            --window[s[right - p.size()] - 'a'];
        }
        if (window == need) {
            answer.push_back(static_cast<int>(right + 1 - p.size()));
        }
    }

    return answer;
}

```

这题容易错在下标。窗口第一次达到目标长度是在 `right + 1 == p.size()`, 起点是 `right + 1 - p.size()`。如果用 `int` 写下标, 要小心 `s.size()` 是无符号类型; 本书示例这里使用 `std::size_t`

控制循环，最终起点再转成 `int`，因为力扣返回 `vector<int>`。

最小覆盖子串是变长窗口里最重要的一题。题目要求在 `s` 中找最短子串，覆盖 `t` 中所有字符及其次数。暴力方法枚举所有子串，再检查是否覆盖，复杂度至少 $O(n^2|\Sigma|)$ 。优化的核心是：右边界扩张直到窗口满足覆盖条件，然后尽可能移动左边界缩短窗口；一旦左移导致窗口不满足，再继续右移。这个过程成立，因为右移只会增加字符，左移只会删除字符，覆盖关系可以被增量维护。

Listing 6.7 最小覆盖子串：变长窗口

```
std::string min_window_substring(const std::string& s, const std::string& t)
{
    constexpr int ASCII_RANGE = 128;
    std::array<int, ASCII_RANGE> need{};
    std::array<int, ASCII_RANGE> window{};
    int required = 0;

    for (const char ch : t) {
        const auto index = static_cast<unsigned char>(ch);
        if (need[index] == 0) {
            ++required;
        }
        ++need[index];
    }

    int formed = 0;
    int left = 0;
    int best_left = 0;
    int best_length = std::numeric_limits<int>::max();

    for (int right = 0; right < static_cast<int>(s.size()); ++right) {
        const auto in = static_cast<unsigned char>(s[right]);
        ++window[in];
        if (need[in] > 0 && window[in] == need[in]) {
            ++formed;
        }

        while (formed == required) {
            const int length = right - left + 1;
            if (length < best_length) {
                best_length = length;
                best_left = left;
            }

            const auto out = static_cast<unsigned char>(s[left]);
            --window[out];
            if (need[out] > 0 && window[out] < need[out]) {
                --formed;
            }
            ++left;
        }
    }

    if (best_length == std::numeric_limits<int>::max()) {
        return "";
    }
    return s.substr(best_left, best_length);
}
```

`required` 表示需要满足的不同字符种类数，`formed` 表示当前窗口里已经满足需求的字符种类数。不能只用窗口总长度判断，因为 `t = "AABC"` 时，两个 `A` 都必须覆盖。复杂度是 $O(n + m + |\Sigma|)$ ，在 ASCII 数组固定时可视为 $O(n + m)$ 。面试表达要说清楚：这题不是固定窗口，因为答案长度未知；它是“先满足，再收缩”的最短覆盖模型。

本章题目复盘模板

数组和窗口题复盘时至少写五句话：暴力枚举什么；慢点是否来自重复扫描、重复统计或候选过多；优化使用了有序性、窗口增量状态还是候选淘汰；左右指针移动条件是什么；每个元素为什么只会被处理常数次数。只写“用双指针”或“用滑动窗口”不算会。

本章力扣主线：11 盛最多水的容器、15 三数之和、209 长度最小的子数组、3 无重复字符的最

长子串、438 找到字符串中所有字母异位词、76 最小覆盖子串。复盘时必须写出窗口不变量和指针移动条件。

第 7 章

哈希表和前缀和

哈希表解决“快速查历史信息”，前缀和解决“快速计算区间信息”。两者结合后，可以把很多子数组问题从 $O(n^2)$ 降到 $O(n)$ 。

两数之和是哈希表入门，但真正要记住的是模型：当前元素需要查询历史中是否存在某个补数。只要查询对象可以作为 key，哈希表就能把线性查找降为平均常数查找。暴力方法枚举 i 和 j ，判断 $\text{nums}[i] + \text{nums}[j] == \text{target}$ ，时间复杂度 $O(n^2)$ 。慢点不在加法，而在“为了每个数反复查找另一个数”。哈希表版本从左到右扫描，用 `value_to_index` 保存已经出现过的值及其下标；处理当前值时，只查 $\text{target} - \text{nums}[i]$ 是否在历史中出现过。

Listing 7.1 两数之和：保存历史值到下标

```
std::vector<int> two_sum_hash(const std::vector<int>& nums, int target)
{
    std::unordered_map<int, int> value_to_index;
    value_to_index.reserve(nums.size());

    for (int index = 0; index < static_cast<int>(nums.size()); ++index) {
        const int need = target - nums[index];
        const auto found = value_to_index.find(need);
        if (found != value_to_index.end()) {
            return {found->second, index};
        }
        value_to_index.emplace(nums[index], index);
    }

    return {};
}
```

这段代码必须先查再插入，避免同一个元素和自己配对。如果数组里有重复值，`emplace` 会保留第一次出现的下标；对力扣 1 这种保证唯一答案的题足够。如果题目要求所有答案，就不能这样提前返回，而要继续扫描并处理去重。面试时可以这样表达：哈希表保存历史值到下标的映射，查询 key 是当前值的补数，平均时间 $O(n)$ ，空间 $O(n)$ 。

前缀和的定义是：`prefix[i]` 表示前 i 个元素的和。那么子数组 `[left, right]` 的和是 `prefix[right + 1] - prefix[left]`。如果题目问和为 k 的子数组个数，就等价于对每个当前位置查找之前有多少个前缀和等于 `current_prefix - k`。

Listing 7.2 和为 K 的子数组

```
int subarray_sum(const std::vector<int>& nums, int k)
{
    std::unordered_map<int, int> prefix_count;
    prefix_count.reserve(nums.size() + 1);
    prefix_count[0] = 1;

    int prefix = 0;
    int answer = 0;
    for (int x : nums) {
        prefix += x;
        const int need = prefix - k;
        if (const auto found = prefix_count.find(need); found != prefix_count.end()) {
            answer += found->second;
        }
        ++prefix_count[prefix];
    }
    return answer;
}
```

为什么先查再插入？因为当前前缀只能和之前的前缀组成非空子数组。如果先插入，在 $k == 0$ 时可能把当前位置和自己配对。为什么保存次数而不是下标？因为题目问个数，一个当前前缀可以

和多个历史前缀组成答案。

最长连续序列展示 `unordered_set` 的另一种用法。把所有数放进集合后，只从没有前驱 $x - 1$ 的数开始扩展。这样每条连续链只从头部扫描一次，每个数最多被访问常数次数，平均时间 $O(n)$ 。

哈希题常见错误包括：用 `operator[]` 做纯查询导致插入多余 `key`；忘记处理重复值；依赖 `unordered_map` 遍历顺序；没有考虑 `long long` 前缀和溢出；把滑动窗口误用到含负数数组。

最长连续序列要求在未排序数组中找最长的连续整数链。排序可以做，复杂度 $O(n \log n)$ ；暴力方法从每个数开始向后找 $x + 1$ 、 $x + 2$ ，如果每次都线性查找，会退化到 $O(n^2)$ 。哈希集合优化了“某个数是否存在”的查询，但如果从每个数都向后扩展，连续链中间的数会被重复扫描。真正的关键是只从链头开始扩展：如果 $x - 1$ 存在，说明 x 不是开头，跳过。

Listing 7.3 最长连续序列：只从链头扩展

```
int longest_consecutive(const std::vector<int>& nums)
{
    std::unordered_set<int> values;
    values.reserve(nums.size());
    for (const int x : nums) {
        values.insert(x);
    }

    int best = 0;
    for (const int x : values) {
        if (values.contains(x - 1)) {
            continue;
        }

        int current = x;
        int length = 1;
        while (values.contains(current + 1)) {
            ++current;
            ++length;
        }
        best = std::max(best, length);
    }

    return best;
}
```

这题的摊还分析要讲清楚：每条连续链只会从最小值开始扫描一次，链中每个元素只在这次扫描里被访问，因此平均时间是 $O(n)$ 。不能在遍历 `values` 时删除元素，除非你非常清楚迭代器失效规则；本书这里选择不删除，代码更直接。

四数相加 II 展示“拆半”的哈希思想。题目给四个数组，统计 $a + b + c + d == 0$ 的元组数量。暴力枚举四个下标是 $O(n^4)$ 。如果固定前三个再查第四个，仍然是 $O(n^3)$ 。更好的做法是把四个数组拆成两组：先统计所有 $a + b$ 的和出现次数，再枚举 $c + d$ ，查询 $-(c + d)$ 出现过多少次。

Listing 7.4 四数相加 II：两两拆分

```
int four_sum_count(const std::vector<int>& nums1,
                  const std::vector<int>& nums2,
                  const std::vector<int>& nums3,
                  const std::vector<int>& nums4)
{
    std::unordered_map<int, int> pair_sum_count;
    pair_sum_count.reserve(nums1.size() * nums2.size());

    for (const int a : nums1) {
        for (const int b : nums2) {
            ++pair_sum_count[a + b];
        }
    }

    int answer = 0;
    for (const int c : nums3) {
        for (const int d : nums4) {
            const auto found = pair_sum_count.find(-(c + d));
            if (found != pair_sum_count.end()) {
                answer += found->second;
            }
        }
    }
}
```

```
    }  
}  
  
return answer;  
}
```

拆半的本质是把 n^4 的组合空间拆成两个 n^2 的集合,再用哈希表连接它们。空间复杂度是 $O(n^2)$ 。这类题的面试表达要说明为什么保存次数:同一个两数和可能由多个下标对产生,每一个都能和后半部分组成不同元组。

哈希题复盘模板

哈希题不要只写“用 map”。复盘时必须写: key 是什么, value 是什么; 查询发生在插入前还是插入后; 重复 key 怎么处理; 是否需要次数、最早位置、最新位置还是集合存在性; 平均复杂度和最坏情况分别是什么。

本章力扣主线: 1 两数之和、49 字母异位词分组、128 最长连续序列、560 和为 K 的子数组、523 连续的子数组和、525 连续数组、454 四数相加 II。复盘时必须说明哈希表保存的是值、下标、次数还是最早位置。

第 8 章

二分查找和答案空间

二分查找的本质不是在数组里找数，而是在单调结构中找边界。只要能定义判断函数 `check(x)`，并且它在答案空间上单调，就可以二分。

基础二分在有序数组中查找目标值。写法看似简单，错误通常来自边界没有排除 `mid`、返回值不清楚或 `left + right` 溢出。

Listing 8.1 基础二分查找

```
int binary_search_index(const std::vector<int>& nums, int target)
{
    int left = 0;
    int right = static_cast<int>(nums.size()) - 1;
    while (left <= right) {
        const int mid = left + (right - left) / 2;
        if (nums[mid] == target) {
            return mid;
        }
        if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}
```

找左边界时，标准库 `lower_bound` 是最好的参照：它找第一个不小于目标的位置。很多题不是找一个等于目标的位置，而是找第一个满足条件的位置。

搜索插入位置就是左边界模型。题目要求如果目标存在，返回下标；如果不存在，返回应该插入的位置。这个位置其实就是第一个 `nums[i] >= target` 的下标。如果所有数都小于目标，答案就是 `nums.size()`。暴力方法从左到右扫描，时间 $O(n)$ ；有序数组给了单调性，所以可以二分。

Listing 8.2 搜索插入位置：lower_bound 写法

```
int search_insert_position(const std::vector<int>& nums, int target)
{
    int left = 0;
    int right = static_cast<int>(nums.size());

    while (left < right) {
        const int mid = left + (right - left) / 2;
        if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }

    return left;
}
```

这里使用左闭右开区间 `[left, right)`。循环不变量是答案一定在这个区间里；当 `nums[mid] < target`，`mid` 以及左侧都不可能是第一个不小于目标的位置，所以令 `left = mid + 1`；否则 `mid` 可能就是答案，不能排除，只能令 `right = mid`。这个“保留 `mid`”是左边界二分的核心。

答案二分把“位置”换成“答案值”。爱吃香蕉的珂珂中，答案是速度。速度越大越容易在规定时间内吃完，所以 `check(speed)` 从 `false` 变成 `true`。我们要找第一个 `true`。分割数组最大值、运输包裹能力、制作花束最少天数也都是这个模型。

Listing 8.3 爱吃香蕉的珂珂：答案二分


```

int min_eating_speed(const std::vector<int>& piles, int hours)
{
    int left = 1;
    int right = *std::max_element(piles.begin(), piles.end());

    auto can_finish = [&](int speed) {
        long long used_hours = 0;
        for (const int pile : piles) {
            used_hours += (pile + speed - 1LL) / speed;
        }
        return used_hours <= hours;
    };

    while (left < right) {
        const int mid = left + (right - left) / 2;
        if (can_finish(mid)) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }

    return left;
}

```

暴力方法可以从速度 1 一直试到最大堆大小，每次计算总小时数，复杂度 $O(mn)$ ，其中 m 是最大速度范围。二分优化来自单调性：速度越大，吃完所需小时数越少；如果某个速度可以完成，那么更大的速度也一定可以完成。我们要找最小可行速度，所以是“第一个 true”。计算向上取整时用 $(pile + speed - 1LL) / speed$ ，其中 1LL 避免整数溢出。

搜索旋转排序数组是另一类二分：单调性不是全局存在，而是每一步至少有一半区间有序。暴力扫描是 $O(n)$ 。二分时先判断 $[left, mid]$ 是否有序；如果有序，再判断目标是否落在这一半里，落在里面就收缩到左半，否则去右半。若左半无序，右半一定有序，用同样逻辑判断。这个题不要死背条件，应该画出旋转数组的两段有序结构。

Listing 8.4 搜索旋转排序数组

```

int search_rotated_array(const std::vector<int>& nums, int target)
{
    int left = 0;
    int right = static_cast<int>(nums.size()) - 1;

    while (left <= right) {
        const int mid = left + (right - left) / 2;
        if (nums[mid] == target) {
            return mid;
        }

        if (nums[left] <= nums[mid]) {
            if (nums[left] <= target && target < nums[mid]) {
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        } else {
            if (nums[mid] < target && target <= nums[right]) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
    }

    return -1;
}

```

这题默认数组没有重复值。如果允许重复，例如 $[1, 0, 1, 1, 1]$ ， $nums[left] == nums[mid]$ 时无法判断哪边有序，可能需要移动边界退化处理，最坏复杂度会变成 $O(n)$ 。面试时主动说明这个前提，会显得你不是机械套模板。

答案二分三步

第一，写出答案范围。第二，写出 `check(x)`。第三，证明单调性，并说明要找最小可行值还是最大可行值。没有单调性，不要二分。

本章力扣主线：704 二分查找、35 搜索插入位置、34 查找第一个和最后一个位置、33 搜索旋转排序数组、162 寻找峰值、875 爱吃香蕉的珂珂、1011 在 D 天内送达包裹的能力、410 分割数组的最大值。

第 9 章

树、图、BFS 和 DFS

树和图题的第一难点是建模。题目未必直接给你图，但只要有状态和状态之间的一步转移，就可以看成图。网格题的格子是节点，相邻方向是边；课程表的课程是节点，先修关系是有向边；单词接龙的单词是节点，一次变化是边。

树是没有环的特殊图。二叉树节点通常只有左右孩子，父子关系天然给出层次；普通图没有固定根，可能有环，必须显式维护访问状态。二者的共同点是：算法不会凭空发生，所有遍历都在回答“当前节点能把什么信息交给相邻节点”。BFS 按层推进，适合无权最短步数、层序遍历和多源扩散；DFS 沿路径推进，适合连通块、路径搜索、状态检测和回溯枚举。本书会讲递归思想，但 C++ 实现优先使用显式队列、栈和循环，因为在线评测里极端深树和大图可能让递归调用栈失控。

为了让代码统一，本章二叉树示例使用下面的节点字段名。力扣原题多使用 `val`，实际提交时把 `value` 改成 `val` 即可，核心逻辑不变。

Listing 9.1 二叉树节点示意

```
struct TreeNode {
    int value = 0;
    TreeNode* left = nullptr;
    TreeNode* right = nullptr;
};
```

二叉树层序遍历是 BFS 的入门题，但它不是“把队列模板背下来”这么简单。题目要求按层返回节点值，因此队列里保存的是下一批待访问节点，而外层循环开始时，队列必须恰好包含当前层的所有节点。暴力做法可以先求树高，再对每一层从根节点重新扫描一次，把深度等于目标层的节点收集出来。这个思路正确，但一棵高度为 h 的树会被重复扫描很多次，最坏退化到 $O(nh)$ 。

优化来自队列的先进先出顺序。根节点先入队，处理当前层时，把它的孩子按从左到右的顺序放到队尾。当前层节点全部弹出后，队列中剩下的正好是下一层节点。这里的关键变量是 `level_size`：它把“当前层的边界”固定下来，避免新入队的下一层节点被提前处理。

Listing 9.2 二叉树层序遍历：固定当前层边界

```
std::vector<std::vector<int>> level_order(TreeNode* root)
{
    std::vector<std::vector<int>> levels;
    if (root == nullptr) {
        return levels;
    }

    std::queue<TreeNode*> queue;
    queue.push(root);

    while (!queue.empty()) {
        const int level_size = static_cast<int>(queue.size());
        std::vector<int> level;
        level.reserve(level_size);

        for (int i = 0; i < level_size; ++i) {
            TreeNode* node = queue.front();
            queue.pop();
            level.push_back(node->value);

            if (node->left != nullptr) {
                queue.push(node->left);
            }
            if (node->right != nullptr) {
                queue.push(node->right);
            }
        }
    }
}
```

```

        levels.push_back(std::move(level));
    }

    return levels;
}

```

这题的复杂度是 $O(n)$ ，每个节点入队一次、出队一次。空间复杂度是 $O(w)$ ， w 是树的最大宽度；如果把输出也计入空间，则是 $O(n)$ 。常见错误有三个：空树忘记返回空数组；没有记录 `level_size`，导致层边界混在一起；把右孩子放到左孩子前面，破坏题目要求的从左到右顺序。面试时要把“不变量”说出来：每次进入外层循环，队列中保存的是同一层、尚未输出的全部节点。

最大深度看起来更像递归题，数学定义也很自然：空树深度为 0，非空树深度是左右子树最大深度加 1。递归写法短，但深度很大的链式树会把调用栈压得很深。用 BFS 写法时，最大深度就是层序遍历处理过的层数；用显式栈写法时，栈里保存节点和它所在深度。两种实现都能避免把深度交给语言调用栈管理。

Listing 9.3 最大深度：显式栈保存节点深度

```

int max_depth(TreeNode* root)
{
    if (root == nullptr) {
        return 0;
    }

    int answer = 0;
    std::vector<std::pair<TreeNode*, int>> stack{{root, 1}};

    while (!stack.empty()) {
        const auto [node, depth] = stack.back();
        stack.pop_back();
        answer = std::max(answer, depth);

        if (node->left != nullptr) {
            stack.emplace_back(node->left, depth + 1);
        }
        if (node->right != nullptr) {
            stack.emplace_back(node->right, depth + 1);
        }
    }

    return answer;
}

```

这段代码的状态非常明确：栈里的每个元素都是一个尚未展开的节点，以及从根到它经过的节点数。暴力和优化的差别不在复杂度，递归和显式栈都是 $O(n)$ ；差别在工程边界。显式栈把潜在的系统调用栈风险变成可控的堆上容器，也更容易扩展成“最大深度路径”“带权深度”“同时记录父节点”等变体。易错点是深度定义：力扣 104 问节点数层数，根节点深度应是 1；如果题目问边数高度，根节点高度才可能记作 0。

验证二叉搜索树经常暴露“只看局部”的错误。很多人会写成：当前节点大于左孩子、小于右孩子，就继续检查左右子树。这个判断不够，因为 BST 的约束是全局的：左子树所有节点都必须小于当前节点，右子树所有节点都必须大于当前节点。暴力补救方法是对每个节点扫描整棵左子树最大值和右子树最小值，但会重复访问，最坏 $O(n^2)$ 。

更稳定的优化是利用 BST 的中序遍历性质。对合法 BST 做中序遍历，访问值必须严格递增。于是全局范围约束被转换成相邻访问值比较：只要当前值不大于前一个中序值，就一定非法。这里要强调“严格递增”，因为力扣 98 不允许重复值出现在左右子树。

Listing 9.4 验证二叉搜索树：显式栈中序遍历

```

bool is_valid_bst(TreeNode* root)
{
    std::vector<TreeNode*> stack;
    TreeNode* current = root;
    std::optional<long long> previous;

    while (current != nullptr || !stack.empty()) {
        while (current != nullptr) {

```

```

        stack.push_back(current);
        current = current->left;
    }

    current = stack.back();
    stack.pop_back();

    if (previous.has_value() && current->value <= previous.value()) {
        return false;
    }
    previous = current->value;
    current = current->right;
}

return true;
}

```

这题的时间复杂度是 $O(n)$ ，空间复杂度是 $O(h)$ ， h 是树高。代码使用 `long long` 保存前一个值，是为了避免节点值接近 `int` 边界时用哨兵值出错；用 `std::optional` 比写一个“极小初值”更严谨。面试表达可以这样组织：先指出局部比较错误，再给出全局约束，最后说明中序遍历把全局约束转成单调序列检查。

最近公共祖先的题意是：给定二叉树中两个节点 p 和 q ，找出最深的那个节点，使它同时是 p 和 q 的祖先。暴力想法是对每个候选节点分别判断它的子树里是否包含 p 和 q ，这会产生大量重复搜索。经典递归解法会让子树向父节点汇报“是否找到了目标”，但同样存在深树调用栈问题。本书给出显式父指针版本：先遍历树，记录每个节点的父节点；再把 p 的所有祖先放入集合；最后从 q 不断向上走，第一次遇到的已访问祖先就是答案。

Listing 9.5 最近公共祖先：父指针和祖先集合

```

TreeNode* lowest_common_ancestor(TreeNode* root, TreeNode* p, TreeNode* q)
{
    if (root == nullptr || p == nullptr || q == nullptr) {
        return nullptr;
    }

    std::unordered_map<TreeNode*, TreeNode*> parent;
    std::vector<TreeNode*> stack{root};
    parent.emplace(root, nullptr);

    while (!stack.empty() && (!parent.contains(p) || !parent.contains(q))) {
        TreeNode* node = stack.back();
        stack.pop_back();

        if (node->left != nullptr) {
            parent.emplace(node->left, node);
            stack.push_back(node->left);
        }
        if (node->right != nullptr) {
            parent.emplace(node->right, node);
            stack.push_back(node->right);
        }
    }

    std::unordered_set<TreeNode*> ancestors;
    TreeNode* current = p;
    while (current != nullptr) {
        ancestors.insert(current);
        current = parent[current];
    }

    current = q;
    while (current != nullptr) {
        if (ancestors.contains(current)) {
            return current;
        }
        current = parent[current];
    }

    return nullptr;
}

```

为什么从 q 向上遇到的第一个公共祖先就是“最近”的？因为向上走的顺序是从深到浅，越早遇到越靠近 q ，同时它已经在 p 的祖先集合中。这个算法时间 $O(n)$ ，空间 $O(n)$ 。如果题目保证 p 和 q 一定在树中，前面的遍历可以在二者父指针都找到后提前停止；如果不保证，就需要完整遍历并检查二者是否出现。常见错误是把“节点值相同”当成“节点相同”，但力扣 236 给的是节点指针，应该比较节点地址。

岛屿数量把网格转成无向图。节点是值为 '1' 的格子，边是上下左右相邻的陆地。题目要求岛屿个数，就是要求连通块个数。暴力思路可能会对每个陆地格子都搜索一次，看它属于哪个岛，这会反复走过同一片陆地。正确优化是：扫描网格时，只要遇到一块尚未访问的陆地，就说明发现了一个新连通块；从它出发把整个连通块标记掉，答案加一。

Listing 9.6 岛屿数量：显式栈标记连通块

```
int num_islands(std::vector<std::vector<char>>& grid)
{
    if (grid.empty() || grid[0].empty()) {
        return 0;
    }

    constexpr char LAND = '1';
    constexpr char WATER = '0';
    constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
        std::pair{1, 0}, std::pair{-1, 0},
        std::pair{0, 1}, std::pair{0, -1}
    };

    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());
    int islands = 0;

    for (int row = 0; row < rows; ++row) {
        for (int col = 0; col < cols; ++col) {
            if (grid[row][col] != LAND) {
                continue;
            }

            ++islands;
            std::vector<std::pair<int, int>> stack{{row, col}};
            grid[row][col] = WATER;

            while (!stack.empty()) {
                const auto [r, c] = stack.back();
                stack.pop_back();

                for (const auto [dr, dc] : DIRECTIONS) {
                    const int next_row = r + dr;
                    const int next_col = c + dc;
                    const bool out_of_bounds =
                        next_row < 0 || next_row >= rows ||
                        next_col < 0 || next_col >= cols;
                    if (out_of_bounds || grid[next_row][next_col] != LAND) {
                        continue;
                    }

                    grid[next_row][next_col] = WATER;
                    stack.emplace_back(next_row, next_col);
                }
            }
        }
    }

    return islands;
}
```

这个实现把访问过的陆地直接改成水，因此不需要额外 **visited** 数组。如果题目不允许修改输入，就用同样大小的布尔矩阵保存访问状态。不变量是：扫描到任意位置时，所有已经发现过的岛都被完全标记为水，尚未被扫描到的陆地如果仍为 '1'，它一定属于一个尚未计数的新岛。时间复杂度 $O(mn)$ ，空间复杂度最坏 $O(mn)$ ，来自显式栈。易错点是访问标记时机：应该在入栈或入队时

立刻标记，而不是弹出后再标记，否则同一格子可能被多个邻居重复加入。

腐烂橘子是多源 BFS。每个新鲜橘子变烂的时间，等于它到最近初始腐烂橘子的最短四联通距离。暴力想法是每一分钟扫描整张网格，看哪些新鲜橘子旁边有腐烂橘子。这种写法虽然直观，却会反复扫描空格、已经腐烂的格子 and 暂时不会变化的格子。优化方法是把所有初始腐烂橘子同时入队，只从当前腐烂边界向外扩散。

Listing 9.7 腐烂橘子：多源 BFS 计算最短腐烂时间

```
int oranges_rotting(std::vector<std::vector<int>>& grid)
{
    if (grid.empty() || grid[0].empty()) {
        return 0;
    }

    constexpr int EMPTY = 0;
    constexpr int FRESH = 1;
    constexpr int ROTTEN = 2;
    constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
        std::pair{1, 0}, std::pair{-1, 0},
        std::pair{0, 1}, std::pair{0, -1}
    };

    const int rows = static_cast<int>(grid.size());
    const int cols = static_cast<int>(grid[0].size());
    std::queue<std::pair<int, int>> queue;
    int fresh_count = 0;

    for (int row = 0; row < rows; ++row) {
        for (int col = 0; col < cols; ++col) {
            if (grid[row][col] == ROTTEN) {
                queue.emplace(row, col);
            } else if (grid[row][col] == FRESH) {
                ++fresh_count;
            }
        }
    }

    int minutes = 0;
    while (!queue.empty() && fresh_count > 0) {
        const int level_size = static_cast<int>(queue.size());
        for (int step = 0; step < level_size; ++step) {
            const auto [row, col] = queue.front();
            queue.pop();

            for (const auto [dr, dc] : DIRECTIONS) {
                const int next_row = row + dr;
                const int next_col = col + dc;
                const bool out_of_bounds =
                    next_row < 0 || next_row >= rows ||
                    next_col < 0 || next_col >= cols;
                if (out_of_bounds || grid[next_row][next_col] != FRESH) {
                    continue;
                }

                grid[next_row][next_col] = ROTTEN;
                --fresh_count;
                queue.emplace(next_row, next_col);
            }
        }
        ++minutes;
    }

    return fresh_count == 0 ? minutes : -1;
}
```

这里 `level_size` 表示同一分钟内会继续扩散的腐烂橘子数量。处理完这一层后，才让 `minutes` 加一。初始没有新鲜橘子时，答案是 0；队列空了但仍有新鲜橘子时，说明有橘子被空格隔开，永远不会腐烂，答案是 -1。代码中 `EMPTY` 没有参与判断，但保留常量能让输入语义更清楚。复杂度仍是 $O(mn)$ ，因为每个格子最多入队一次。

课程表是有向图判环。每门课是节点，先修关系 `[course, prerequisite]` 表示必须先学

`prerequisite`，再学 `course`，所以边的方向应是 `prerequisite -> course`。暴力想法是反复扫描所有课程，找当前没有未完成先修课的课程；每学一门课又重新检查所有依赖，容易写成 $O(nm)$ 。拓扑排序的优化点是入度数组：`indegree[x]` 表示课程 `x` 还有多少前置依赖没有被满足。

Listing 9.8 课程表 II：拓扑排序输出可行学习顺序

```
std::vector<int> find_order(
    int num_courses,
    const std::vector<std::vector<int>>& prerequisites)
{
    std::vector<std::vector<int>> graph(num_courses);
    std::vector<int> indegree(num_courses, 0);

    for (const auto& edge : prerequisites) {
        const int course = edge[0];
        const int prerequisite = edge[1];
        graph[prerequisite].push_back(course);
        ++indegree[course];
    }

    std::queue<int> queue;
    for (int course = 0; course < num_courses; ++course) {
        if (indegree[course] == 0) {
            queue.push(course);
        }
    }

    std::vector<int> order;
    order.reserve(num_courses);

    while (!queue.empty()) {
        const int course = queue.front();
        queue.pop();
        order.push_back(course);

        for (const int next : graph[course]) {
            --indegree[next];
            if (indegree[next] == 0) {
                queue.push(next);
            }
        }
    }

    if (static_cast<int>(order.size()) != num_courses) {
        return {};
    }
    return order;
}
```

这段代码同时覆盖力扣 207 和 210。207 只问能否完成所有课程，可以返回 `order.size() == num_courses`；210 要返回一个具体顺序，若有环则返回空数组。正确性的核心是：入度为 0 的节点当前没有任何未满足依赖，可以安全学习；学习它只会减少它指向的后继课程入度，不会影响其他边。每个节点最多入队一次，每条边处理一次，复杂度 $O(n + m)$ 。最常见错误是把边方向建反。边建反时，有时仍然能判断是否有环，但输出顺序会变成反向，面试中必须主动说清方向。

单词接龙是无权图最短路。节点是单词，两个单词只差一个字符就连一条边；答案是起点词到终点词的最短转换长度。直接建图的暴力方法是枚举所有单词对，逐字符比较是否只差一位，复杂度约为 $O(n^2L)$ ，其中 L 是单词长度。BFS 本身没有问题，慢在找邻居。

优化思路是用通配模式把邻居查询变成哈希查询。比如 `hot` 可以生成 `*ot`、`h*t`、`ho*` 三个模式；所有能匹配同一模式的单词，彼此之间只差一个字符。预处理时把模式映射到单词列表，BFS 时对当前单词生成模式，从桶里取候选邻居。这样避免了每一步都扫描全词表。

Listing 9.9 单词接龙：通配模式加 BFS

```
int ladder_length(const std::string& begin_word,
                 const std::string& end_word,
                 const std::vector<std::string>& word_list)
{
    constexpr char WILDCARD = '*';
```



```

const int word_length = static_cast<int>(begin_word.size());

std::unordered_set<std::string> dictionary(
    word_list.begin(), word_list.end());
if (!dictionary.contains(end_word)) {
    return 0;
}

std::unordered_map<std::string, std::vector<std::string>> buckets;
for (const std::string& word : word_list) {
    for (int i = 0; i < word_length; ++i) {
        std::string pattern = word;
        pattern[i] = WILDCARD;
        buckets[pattern].push_back(word);
    }
}

std::queue<std::pair<std::string, int>> queue;
std::unordered_set<std::string> visited;
queue.emplace(begin_word, 1);
visited.insert(begin_word);

while (!queue.empty()) {
    const auto [word, distance] = queue.front();
    queue.pop();
    if (word == end_word) {
        return distance;
    }

    for (int i = 0; i < word_length; ++i) {
        std::string pattern = word;
        pattern[i] = WILDCARD;
        auto found = buckets.find(pattern);
        if (found == buckets.end()) {
            continue;
        }

        for (const std::string& next : found->second) {
            if (visited.contains(next)) {
                continue;
            }
            visited.insert(next);
            queue.emplace(next, distance + 1);
        }
        found->second.clear();
    }
}

return 0;
}

```

这题的 BFS 不变量是：队列中每个状态的 **distance** 都是从起点到该单词的最短转换长度；第一次到达 **endWord** 时，答案已经最短。为什么可以在访问后清空模式桶？因为一个模式桶只需要被展开一次。之后再遇到同一模式，桶里的所有邻居要么已经访问，要么已经入队；重复扫描只会增加时间。预处理复杂度约为 $O(nL^2)$ ，因为每次生成模式会复制长度为 L 的字符串；BFS 展开在清桶优化后也是同量级。若要继续优化，可以把单词编号化，模式只保存编号，但面试中这个版本已经足够清晰。

树图题复盘模板

每道树图题都先写四句话：节点是什么，边是什么，状态里保存什么，访问标记何时设置。二叉树题再补一句返回值语义或队列不变量；图题再补一句为什么 BFS 第一次到达就是最短，或者为什么 DFS 标记完整连通块。写完代码后，用空输入、单节点、链式树、完全树、孤立节点、有环依赖、不可达终点各测一遍。能把这些边界说清楚，才算真正掌握，而不是只背了遍历模板。

回溯、枚举和搜索空间剪枝

回溯题表面上是在“试所有可能”，本质上是在一棵决策树上搜索。每一层做一个选择，每条从根到叶子的路径代表一个候选答案。真正要学会的不是某个递归模板，而是四个问题：当前层决定什么，已经选择了什么，下一步还能选什么，什么时候可以停止或剪枝。只要这四件事说不清，代码再短也只是碰巧通过样例。

暴力枚举和回溯的边界并不绝对。最朴素的暴力会把所有组合全部生成出来，再逐个检查是否合法；回溯则把“合法性判断”提前到生成过程中。一旦发现当前前缀已经不可能形成答案，就不再向下扩展。这个动作叫剪枝。剪枝不是为了让代码看起来高级，而是减少决策树里根本不需要访问的分支。

C++面试题常用递归表达回溯，因为调用栈天然保存了“走到第几层、下一步尝试哪个分支、回退后恢复什么状态”。但递归深度不受控时会带来工程风险。本章讲递归心智模型，代码尽量把栈帧显式写出来，让搜索过程、状态恢复和边界条件更可见。这样写会比递归略长，但它能训练你看清楚回溯真正维护的状态。

全排列是回溯的第一道标准题。题目给定互不相同的数字，要求返回所有排列。暴力想法是枚举长度为 n 的所有序列，再检查是否每个数字恰好使用一次。若把每个位置都看成有 n 种选择，总分支数是 n^n ，绝大多数序列会因为重复使用元素而被丢弃。优化方法是在生成过程中维护 `used` 数组，保证同一个下标不会被选两次。这样搜索树只保留真正的排列分支，叶子数变成 $n!$ 。

下面的实现用显式栈模拟递归。每个栈帧保存当前层接下来要尝试的候选下标 `next_index`。进入下一层时，把选择加入 `path` 并标记 `used`；一层候选全部尝试完时，弹出栈帧，并撤销上一层做出的选择。

Listing 10.1 全排列：显式栈模拟回溯

```
std::vector<std::vector<int>> permute(const std::vector<int>& nums)
{
    struct Frame {
        int next_index = 0;
    };

    const int n = static_cast<int>(nums.size());
    std::vector<std::vector<int>> answer;
    std::vector<int> path;
    std::vector<char> used(n, false);
    std::vector<Frame> stack{{0}};

    while (!stack.empty()) {
        if (static_cast<int>(path.size()) == n) {
            answer.push_back(path);
            stack.pop_back();
            if (!path.empty()) {
                const int last_value = path.back();
                path.pop_back();
                const auto found = std::find(nums.begin(), nums.end(), last_value);
                used[static_cast<int>(found - nums.begin())] = false;
            }
            continue;
        }

        Frame& frame = stack.back();
        while (frame.next_index < n && used[frame.next_index]) {
            ++frame.next_index;
        }
        if (frame.next_index == n) {
            stack.pop_back();
            if (!path.empty()) {
                const int last_value = path.back();
                path.pop_back();
            }
        }
    }
}
```

```

        const auto found = std::find(nums.begin(), nums.end(), last_value);
        used[static_cast<int>(found - nums.begin())] = false;
    }
    continue;
}

const int chosen_index = frame.next_index;
++frame.next_index;
used[chosen_index] = true;
path.push_back(nums[chosen_index]);
stack.push_back(Frame{0});
}

return answer;
}

```

这段代码为了突出栈帧，撤销选择时用值反查下标；在生产代码里可以把“本层选择的下标”也放进下一层栈帧，避免反查。它的时间复杂度必须按输出规模理解：一共有 $n!$ 个排列，每个排列复制长度为 n 的路径，所以输出成本是 $O(n \cdot n!)$ 。额外空间是路径、标记和栈，都是 $O(n)$ ，不计答案。复盘这题时要能说清楚：**used** 是路径合法性的约束，**path.size() == n** 是叶子条件，回退必须撤销最后一次选择。

如果输入含重复数字，全排列 II 的难点就变成去重。很多人会先生成所有排列，再放进 **set** 去重；这当然能做，但浪费大量重复分支。更好的方法是先排序，然后在同一层跳过“相同数值且前一个相同值还没有被本路径使用”的候选。这个条件的含义是：同一层里，相同值只允许最左边那个代表这一类选择，避免生成形如交换两个相同值位置的重复排列。

Listing 10.2 全排列 II：排序后按层去重

```

std::vector<std::vector<int>> permute_unique(std::vector<int> nums)
{
    std::sort(nums.begin(), nums.end());

    const int n = static_cast<int>(nums.size());
    std::vector<std::vector<int>> answer;
    std::vector<int> path;
    std::vector<char> used(n, false);

    struct State {
        std::vector<int> path;
        std::vector<char> used;
    };

    std::vector<State> stack{{path, used}};
    while (!stack.empty()) {
        State state = std::move(stack.back());
        stack.pop_back();

        if (static_cast<int>(state.path.size()) == n) {
            answer.push_back(std::move(state.path));
            continue;
        }

        for (int i = n - 1; i >= 0; --i) {
            if (state.used[i]) {
                continue;
            }
            if (i > 0 && nums[i] == nums[i - 1] && !state.used[i - 1]) {
                continue;
            }

            State next = state;
            next.used[i] = true;
            next.path.push_back(nums[i]);
            stack.push_back(std::move(next));
        }
    }

    return answer;
}

```

这个版本为了让去重条件清楚，直接把状态复制进栈，适合教材解释；高性能版本会用一份路径和标记做原地回退。去重条件最容易写反。排序后，相同数字挨在一起，`!state.used[i - 1]` 表示前一个相同数字在当前层还没有被拿来作为代表，此时选择后一个会制造重复分支，所以跳过。时间复杂度仍然与唯一排列数量成正比，最坏不超过 $O(n \cdot n!)$ 。

子集题的决策树更简单。每个元素只有两种选择：选或者不选。暴力枚举可以用二进制掩码，从 0 到 $2^n - 1$ ，第 i 位表示是否选择第 i 个元素。回溯写法则是沿着数组位置向后决定。两种方法本质相同，都是遍历大小为 2^n 的搜索空间。

Listing 10.3 子集：迭代扩展已有答案

```
std::vector<std::vector<int>> subsets(const std::vector<int>& nums)
{
    std::vector<std::vector<int>> answer{{}};

    for (const int x : nums) {
        const int old_size = static_cast<int>(answer.size());
        for (int i = 0; i < old_size; ++i) {
            std::vector<int> next = answer[i];
            next.push_back(x);
            answer.push_back(std::move(next));
        }
    }

    return answer;
}
```

这段代码的不变量是：处理完前 i 个元素后，`answer` 包含这些元素的全部子集。加入新元素 x 时，旧子集保持“不选 x ”的版本；每个旧子集复制一份并加上 x ，得到“选 x ”的版本。这个算法没有剪枝，因为所有子集都是合法答案。时间复杂度 $O(n2^n)$ ，空间复杂度同样由输出规模主导。

组合总和比子集多了一个目标约束。题目给定候选数组和目标值，要求找出所有和为目标值的组合，同一个数字可以重复使用。暴力思路是不断选数字，直到长度达到某个上限再检查和；问题是上限不好定，而且大量分支会在总和已经超过目标后继续扩展。优化来自两个事实：候选数字都是正数；排序后，如果当前候选已经大于剩余目标，后面的更大候选也不可能使用。

Listing 10.4 组合总和：显式状态保存起点和剩余目标

```
std::vector<std::vector<int>> combination_sum(std::vector<int> candidates,
                                              int target)
{
    std::sort(candidates.begin(), candidates.end());

    struct State {
        int start = 0;
        int remaining = 0;
        std::vector<int> path;
    };

    std::vector<std::vector<int>> answer;
    std::vector<State> stack{{0, target, {}}};

    while (!stack.empty()) {
        State state = std::move(stack.back());
        stack.pop_back();

        if (state.remaining == 0) {
            answer.push_back(std::move(state.path));
            continue;
        }

        for (int i = static_cast<int>(candidates.size()) - 1;
             i >= state.start;
             --i) {
            if (candidates[i] > state.remaining) {
                continue;
            }
            State next = state;
            next.path.push_back(candidates[i]);
            next.remaining -= candidates[i];
        }
    }
}
```

```

        next.start = i;
        stack.push_back(std::move(next));
    }
}

return answer;
}

```

start 的语义很重要：组合不关心顺序， $[2,3,2]$ 和 $[2,2,3]$ 是同一个答案，因此后续只能从当前下标及其右侧继续选，避免同一组合以不同顺序重复出现。因为允许重复使用当前数字，下一层的 **start** 仍然是 **i**；如果题目变成每个数字只能用一次，就应设为 $i + 1$ 。这就是组合类题最核心的变体开关。复杂度取决于目标值和候选分布，不能简单写成多项式；面试里应说明它是指数级搜索，但剪枝能显著减少无效分支。

括号生成的关键不是枚举所有长度为 $2n$ 的括号串再检查。那样有 2^{2n} 个候选，大多数不合法。合法前缀必须满足两个计数约束：左括号数量不能超过 n ；任何前缀中右括号数量不能超过左括号数量。把这两个约束提前放进生成过程，就不会产生明显非法的前缀。

Listing 10.5 括号生成：前缀合法性剪枝

```

std::vector<std::string> generate_parenthesis(int n)
{
    struct State {
        std::string text;
        int left = 0;
        int right = 0;
    };

    std::vector<std::string> answer;
    std::vector<State> stack{{"", 0, 0}};

    while (!stack.empty()) {
        State state = std::move(stack.back());
        stack.pop_back();

        if (static_cast<int>(state.text.size()) == 2 * n) {
            answer.push_back(std::move(state.text));
            continue;
        }

        if (state.right < state.left) {
            State next = state;
            next.text.push_back(')');
            ++next.right;
            stack.push_back(std::move(next));
        }
        if (state.left < n) {
            State next = state;
            next.text.push_back('(');
            ++next.left;
            stack.push_back(std::move(next));
        }
    }

    return answer;
}

```

这题的正确性可以用前缀不变量解释：任何时刻 $left \leq n$ 且 $right \leq left$ 。如果 $right > left$ ，说明某个右括号没有左括号匹配，后面再加字符也无法修复这个非法前缀；如果 $left > n$ ，也已经超过题目允许。输出数量是第 n 个卡特兰数，算法复杂度至少与答案数量成正比。易错点是终止条件：必须长度达到 $2n$ 才能加入答案，只看 $left == right$ 会把短前缀误加进去。

单词搜索把回溯放进网格图。节点是格子，边是上下左右相邻，路径不能重复使用同一个格子。暴力想法是从每个格子出发尝试所有方向，最后检查路径字符串；优化点是边走边比较目标单词的当前位置，一旦字符不匹配立即停止。访问标记也必须属于当前路径，而不是全局永久标记，因为同一个格子可以出现在不同起点或不同路径中。

Listing 10.6 单词搜索：显式栈保存路径访问状态

```

bool exist(std::vector<std::vector<char>>& board, const std::string& word)
{
    if (word.empty()) {
        return true;
    }
    if (board.empty() || board[0].empty()) {
        return false;
    }

    constexpr std::array<std::pair<int, int>, 4> DIRECTIONS{
        std::pair{1, 0}, std::pair{-1, 0},
        std::pair{0, 1}, std::pair{0, -1}
    };

    const int rows = static_cast<int>(board.size());
    const int cols = static_cast<int>(board[0].size());

    struct State {
        int row = 0;
        int col = 0;
        int index = 0;
        std::vector<std::vector<char>> visited;
    };

    for (int row = 0; row < rows; ++row) {
        for (int col = 0; col < cols; ++col) {
            if (board[row][col] != word[0]) {
                continue;
            }

            std::vector<std::vector<char>> visited(rows, std::vector<char>(cols, false));
            visited[row][col] = true;
            std::vector<State> stack{{row, col, 0, std::move(visited)}};

            while (!stack.empty()) {
                State state = std::move(stack.back());
                stack.pop_back();
                if (state.index + 1 == static_cast<int>(word.size())) {
                    return true;
                }

                for (const auto [dr, dc] : DIRECTIONS) {
                    const int next_row = state.row + dr;
                    const int next_col = state.col + dc;
                    const int next_index = state.index + 1;
                    const bool out_of_bounds =
                        next_row < 0 || next_row >= rows ||
                        next_col < 0 || next_col >= cols;
                    if (out_of_bounds || state.visited[next_row][next_col] ||
                        board[next_row][next_col] != word[next_index]) {
                        continue;
                    }

                    State next = state;
                    next.row = next_row;
                    next.col = next_col;
                    next.index = next_index;
                    next.visited[next_row][next_col] = true;
                    stack.push_back(std::move(next));
                }
            }
        }
    }

    return false;
}

```

这个教材版把 **visited** 放进状态里，语义最清楚，但会复制较多布尔矩阵；原地回退版性能更好。本题最坏复杂度接近 $O(mn \cdot 3^L)$ ：第一步最多从每个格子出发，之后每一步通常不能回到上一个格子，所以分支近似 3， L 是单词长度。常见错误是把访问标记设成全局，导致某个格子被一条失败路径占用后，其他合法路径无法使用它。

N 皇后是回溯剪枝的代表。暴力方法在 $n \times n$ 棋盘上选 n 个格子，再检查行、列、对角线是否冲突，搜索空间巨大。优化方式是按行放皇后：第 row 层只决定第 row 行皇后放在哪一列。这样行冲突天然消失，只需要维护列、主对角线和副对角线是否被占用。

Listing 10.7 N 皇后：按行放置并维护三类占用

```
std::vector<std::vector<std::string>> solve_n_queens(int n)
{
    struct State {
        int row = 0;
        std::vector<int> queens;
        std::vector<char> cols;
        std::vector<char> diag1;
        std::vector<char> diag2;
    };

    std::vector<std::vector<std::string>> answer;
    std::vector<State> stack{{0,
        {},
        std::vector<char>(n, false),
        std::vector<char>(2 * n - 1, false),
        std::vector<char>(2 * n - 1, false)}};

    while (!stack.empty()) {
        State state = std::move(stack.back());
        stack.pop_back();

        if (state.row == n) {
            std::vector<std::string> board(n, std::string(n, '.'));
            for (int row = 0; row < n; ++row) {
                board[row][state.queens[row]] = 'Q';
            }
            answer.push_back(std::move(board));
            continue;
        }

        for (int col = n - 1; col >= 0; --col) {
            const int main_diag = state.row - col + n - 1;
            const int anti_diag = state.row + col;
            if (state.cols[col] || state.diag1[main_diag] ||
                state.diag2[anti_diag]) {
                continue;
            }

            State next = state;
            next.queens.push_back(col);
            next.cols[col] = true;
            next.diag1[main_diag] = true;
            next.diag2[anti_diag] = true;
            ++next.row;
            stack.push_back(std::move(next));
        }
    }

    return answer;
}
```

主对角线上的格子满足 $\text{row} - \text{col}$ 相同；为了把负数下标变成数组下标，代码加上 $n - 1$ 。副对角线满足 $\text{row} + \text{col}$ 相同，范围天然就是 0 到 $2n - 2$ 。这题的面试重点不是把棋盘画出来，而是把冲突检测从扫描整张棋盘优化成三个数组的常数查询。复杂度仍然是指数级，但剪枝把不可行分支尽早挡在下一层之外。

回溯题复盘模板

回溯题不要先背代码。先画出决策树的一层，写清本层选择、路径状态、可选集合、终止条件和剪枝条件。组合题重点看 **start** 是否允许重复使用当前元素；排列题重点看 **used** 和同层去重；网格题重点看访问标记是路径级还是全局级；括号和棋盘题重点把非法前缀提前剪掉。最后再问一句：答案数量本身有多大？如果答案就是指数级，复杂度不可能被写成线性或多项式。

第 11 章

动态规划：状态、转移和重复子问题

动态规划不是“看到最值就开数组”。它解决的是有重叠子问题、且大问题能由小问题稳定推出的问题。暴力搜索之所以慢，是因为同一个子问题被不同路径反复计算；动态规划的优化，就是给子问题命名、保存结果，并按照依赖顺序把结果推出来。学 DP 最重要的五件事是：状态表示什么，答案在哪里，初始状态是什么，转移从哪里来，遍历顺序为什么满足依赖。

爬楼梯是最小的 DP 模型。题目问到达第 n 阶有多少种方法，每次可以爬 1 或 2 阶。暴力递归会写成 $\text{ways}(n) = \text{ways}(n - 1) + \text{ways}(n - 2)$ ，但 $\text{ways}(n - 2)$ 、 $\text{ways}(n - 3)$ 等子问题会被反复计算，形成指数级时间。优化方法是从小到大保存每个阶数的答案。

Listing 11.1 爬楼梯：滚动变量保存最近两个状态

```
int climb_stairs(int n)
{
    if (n <= 2) {
        return n;
    }

    int previous_two = 1;
    int previous_one = 2;
    for (int step = 3; step <= n; ++step) {
        const int current = previous_one + previous_two;
        previous_two = previous_one;
        previous_one = current;
    }

    return previous_one;
}
```

这里状态可以写成 $\text{dp}[i]$ ：到达第 i 阶的方法数。最后一步要么从 $i - 1$ 走 1 阶，要么从 $i - 2$ 走 2 阶，因此转移是 $\text{dp}[i] = \text{dp}[i - 1] + \text{dp}[i - 2]$ 。由于每个状态只依赖前两个状态，可以把数组压成两个变量。时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 。这题的价值不在难度，而在于展示 DP 的完整骨架。

打家劫舍把“相邻不能同时选”的约束放进状态。暴力方法是对每间房决定偷或不偷，枚举 2^n 个子集，再过滤相邻冲突。更好的建模是让 $\text{dp}[i]$ 表示考虑前 i 间房能偷到的最大金额。第 i 间房只有两种结局：不偷它，答案是 $\text{dp}[i - 1]$ ；偷它，就不能偷第 $i - 1$ 间，答案是 $\text{dp}[i - 2] + \text{nums}[i - 1]$ 。

Listing 11.2 打家劫舍：选与不选的状态转移

```
int rob(const std::vector<int>& nums)
{
    int previous_two = 0;
    int previous_one = 0;

    for (const int money : nums) {
        const int current = std::max(previous_one, previous_two + money);
        previous_two = previous_one;
        previous_one = current;
    }

    return previous_one;
}
```

这段代码把 $\text{dp}[i - 2]$ 和 $\text{dp}[i - 1]$ 压成两个变量。遍历到当前房屋时， previous_one 是不偷当前房屋时可继承的最优值， $\text{previous_two} + \text{money}$ 是偷当前房屋时的最优值。正确性来自最优子结构：当前决策之后，剩余可用的历史范围只与前一间或前两间有关，不需要知道更早的具体选择。易错点是下标：如果用数组，建议定义 $\text{dp}[0] = 0$ ， $\text{dp}[1] = \text{nums}[0]$ ，这样第 i 个状态表

示前 i 间房，避免把房屋下标和状态下标混在一起。

零钱兑换展示“最少数量”型 DP。题目给硬币面额和目标金额，要求凑成目标金额所需的最少硬币数。暴力搜索会尝试所有硬币序列，许多不同顺序会到达同一个剩余金额，例如先选 1 再选 2，与先选 2 再选 1，都可能进入同一个子问题。状态应定义为 $dp[amount]$ ：凑出金额 $amount$ 的最少硬币数。

Listing 11.3 零钱兑换：从可达金额推出新金额

```
int coin_change(const std::vector<int>& coins, int amount)
{
    constexpr int INF = 1'000'000'000;
    std::vector<int> dp(amount + 1, INF);
    dp[0] = 0;

    for (int value = 1; value <= amount; ++value) {
        for (const int coin : coins) {
            if (coin > value || dp[value - coin] == INF) {
                continue;
            }
            dp[value] = std::min(dp[value], dp[value - coin] + 1);
        }
    }

    return dp[amount] == INF ? -1 : dp[amount];
}
```

转移的含义是：如果最后一枚硬币是 $coin$ ，那么前面必须先凑出 $value - coin$ ，所以候选答案是 $dp[value - coin] + 1$ 。这里用一个足够大的 INF 表示不可达，而不是用 -1 参与最小值计算，因为 $-1 + 1$ 会污染转移。时间复杂度 $O(amount \cdot c)$ ， c 是硬币种类数。完全背包类题还要区分“组合数”和“排列数”：本题只问最少硬币数，内外循环顺序不会影响最优值；如果问方案数量，循环顺序就会改变计数语义。

最长递增子序列有两种经典解法。第一种 DP 定义 $dp[i]$ 为以 $nums[i]$ 结尾的最长递增子序列长度。为了求它，需要看所有 $j < i$ 且 $nums[j] < nums[i]$ 的位置，转移为 $dp[i] = \max(dp[j] + 1)$ 。这比枚举所有子序列的 2^n 好很多，但仍是 $O(n^2)$ 。

Listing 11.4 最长递增子序列：二次 DP

```
int length_of_lis_dp(const std::vector<int>& nums)
{
    if (nums.empty()) {
        return 0;
    }

    std::vector<int> dp(nums.size(), 1);
    int answer = 1;

    for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
        for (int j = 0; j < i; ++j) {
            if (nums[j] < nums[i]) {
                dp[i] = std::max(dp[i], dp[j] + 1);
            }
        }
        answer = std::max(answer, dp[i]);
    }

    return answer;
}
```

第二种优化来自“只关心长度对应的最小结尾”。维护数组 $tails$ ，其中 $tails[len - 1]$ 表示长度为 len 的递增子序列中，最小可能结尾值。结尾越小，后面越容易接上更大的数。遍历新数 x 时，用二分找到第一个大于等于 x 的位置并替换；如果找不到，说明 x 可以扩展出更长序列。

Listing 11.5 最长递增子序列：贪心尾值加二分

```
int length_of_lis_binary_search(const std::vector<int>& nums)
{
    std::vector<int> tails;
```

```

    for (const int x : nums) {
        auto position = std::lower_bound(tails.begin(), tails.end(), x);
        if (position == tails.end()) {
            tails.push_back(x);
        } else {
            *position = x;
        }
    }

    return static_cast<int>(tails.size());
}

```

这个算法经常被误解成 `tails` 本身就是最终的子序列。它不一定是。它保存的是每个长度下最有潜力的最小结尾。替换操作不会减少已经找到的长度，只会让同长度的结尾更小，从而给未来元素更多机会。时间复杂度 $O(n \log n)$ ，空间复杂度 $O(n)$ 。如果面试官追问为什么能替换，要回答：相同长度下，更小结尾不会比更大结尾更差，因为它能接纳的后续元素集合只会更多。

单词拆分把字符串前缀变成状态。题目给字符串和词典，问字符串能否被切成若干词典单词。暴力搜索会尝试每个切分点，遇到相同后缀时重复判断。状态 `dp[i]` 表示前 `i` 个字符是否可以被合法拆分。若存在 `j < i`，使得 `dp[j]` 为真且子串 `s[j..i)` 在词典中，则 `dp[i]` 为真。

Listing 11.6 单词拆分：前缀可达状态

```

bool word_break(const std::string& s, const std::vector<std::string>& word_dict)
{
    std::unordered_set<std::string> words(word_dict.begin(), word_dict.end());
    std::vector<char> dp(s.size() + 1, false);
    dp[0] = true;

    for (std::size_t right = 1; right <= s.size(); ++right) {
        for (std::size_t left = 0; left < right; ++left) {
            if (!dp[left]) {
                continue;
            }
            if (words.contains(s.substr(left, right - left))) {
                dp[right] = true;
                break;
            }
        }
    }

    return dp[s.size()];
}

```

这里 `dp[0] = true` 代表空前缀可拆分，是所有从 0 开始的单词能够成立的基础。代码直接使用 `substr`，写法清晰，但会复制子串；如果输入规模更大，可以限制最大单词长度，或者用字符串视图和自定义哈希、字典树来减少复制。复杂度通常写成 $O(n^3)$ 的保守形式，因为有两层切分枚举，子串构造和哈希也与长度有关；若把子串查询视作均摊常数，则是 $O(n^2)$ 。面试中最好主动说明这个实现的复制成本。

最长公共子序列是二维 DP 的代表。子序列可以删除字符但不能改变相对顺序。暴力方法会枚举两个字符串的所有子序列再比较，数量指数级。状态 `dp[i][j]` 表示 `text1` 的前 `i` 个字符和 `text2` 的前 `j` 个字符的最长公共子序列长度。

Listing 11.7 最长公共子序列：二维前缀 DP

```

int longest_common_subsequence(const std::string& text1,
                               const std::string& text2)
{
    const int rows = static_cast<int>(text1.size());
    const int cols = static_cast<int>(text2.size());
    std::vector<std::vector<int>> dp(rows + 1, std::vector<int>(cols + 1, 0));

    for (int i = 1; i <= rows; ++i) {
        for (int j = 1; j <= cols; ++j) {
            if (text1[i - 1] == text2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {

```

```

        dp[i][j] = std::max(dp[i - 1][j], dp[i][j - 1]);
    }
}

return dp[rows][cols];
}

```

为什么相等时来自左上角？因为当前两个字符可以配成公共子序列的最后一个字符，剩下问题变成两个前缀各去掉这个字符。为什么不等时取上方和左方最大值？因为两个末尾字符不能同时作为同一个结尾，至少要丢掉其中一个。多出来的第 0 行和第 0 列代表空串，避免了边界分支。时间复杂度 $O(mn)$ ，空间复杂度 $O(mn)$ ；如果只要长度，可以滚动数组压到 $O(n)$ 。

编辑距离比 LCS 更综合。题目允许插入、删除、替换，把一个单词变成另一个单词，求最少操作数。状态 $dp[i][j]$ 表示把 `word1` 的前 i 个字符变成 `word2` 的前 j 个字符的最少操作数。这个定义必须精确，因为三种操作都围绕“前缀末尾”展开。

Listing 11.8 编辑距离：插入、删除、替换三种转移

```

int min_distance(const std::string& word1, const std::string& word2)
{
    const int rows = static_cast<int>(word1.size());
    const int cols = static_cast<int>(word2.size());
    std::vector<std::vector<int>> dp(rows + 1, std::vector<int>(cols + 1, 0));

    for (int i = 0; i <= rows; ++i) {
        dp[i][0] = i;
    }
    for (int j = 0; j <= cols; ++j) {
        dp[0][j] = j;
    }

    for (int i = 1; i <= rows; ++i) {
        for (int j = 1; j <= cols; ++j) {
            if (word1[i - 1] == word2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1];
                continue;
            }

            const int delete_cost = dp[i - 1][j] + 1;
            const int insert_cost = dp[i][j - 1] + 1;
            const int replace_cost = dp[i - 1][j - 1] + 1;
            dp[i][j] = std::min({delete_cost, insert_cost, replace_cost});
        }
    }

    return dp[rows][cols];
}

```

初始化含义是：把长度为 i 的前缀变成空串，需要删除 i 次；把空串变成长度为 j 的前缀，需要插入 j 次。若末尾字符相同，不需要新增操作，直接继承左上角；若不同，删除对应上方，插入对应左方，替换对应左上角。时间和空间都是 $O(mn)$ 。易错点是从哪个字符串视角解释插入和删除：只要状态定义固定， $dp[i - 1][j] + 1$ 就是删掉 `word1` 的末尾， $dp[i][j - 1] + 1$ 就是在 `word1` 末尾插入 `word2[j - 1]`。

动态规划题经常可以做空间优化，但不要为了炫技先写滚动数组。二维表能展示依赖关系，滚动数组则要求你非常清楚哪些旧值还要用、哪些可以覆盖。面试时，先给出清晰状态和二维实现，再说明可压缩空间，通常比一上来写难读的一维代码更稳。

动态规划题复盘模板

每道 DP 题按五步复盘：第一，状态是什么，必须能用自然语言解释；第二，答案在 $dp[n]$ 、 $dp[m][n]$ 还是某个最大值里；第三，空状态和最小状态如何初始化；第四，转移枚举的是最后一步、最后一个选择，还是最后一段区间；第五，遍历顺序是否保证依赖状态已经算完。若你只能背出公式，却说不出公式对应的最后一步选择，这题就还没有真正掌握。

力扣刷题手册

刷题不要按编号顺序推进，要按题型推进。每道题都记录题型、输入规模、暴力解、优化来源、不变量、使用的数据结构、复杂度、错因和一周后是否能独立重写。

数组和字符串主线：1 两数之和、26 删除有序数组重复项、27 移除元素、88 合并两个有序数组、53 最大子数组和、283 移动零、344 反转字符串。

双指针和窗口主线：3 无重复字符的最长子串、11 盛最多水的容器、15 三数之和、42 接雨水、76 最小覆盖子串、209 长度最小的子数组、438 找到字符串中所有字母异位词。

哈希和前缀和主线：49 字母异位词分组、128 最长连续序列、202 快乐数、560 和为 K 的子数组、523 连续的子数组和、525 连续数组。

栈和堆主线：20 有效括号、155 最小栈、739 每日温度、84 柱状图中最大的矩形、215 数组中的第 K 个最大元素、347 前 K 个高频元素、295 数据流的中位数。

二分主线：35 搜索插入位置、34 在排序数组中查找元素的第一个和最后一个位置、33 搜索旋转排序数组、162 寻找峰值、875 爱吃香蕉的珂珂、410 分割数组的最大值。

树和图主线：102 二叉树层序遍历、104 最大深度、98 验证二叉搜索树、236 最近公共祖先、200 岛屿数量、994 腐烂的橘子、207 课程表、210 课程表 II、127 单词接龙。

回溯主线：46 全排列、47 全排列 II、78 子集、39 组合总和、22 括号生成、79 单词搜索、51 N 皇后。

动态规划主线：70 爬楼梯、198 打家劫舍、322 零钱兑换、300 最长递增子序列、139 单词拆分、1143 最长公共子序列、72 编辑距离。

如果一道题三次都需要看题解，不要继续刷新题，先回到它所属章节，重新写暴力到优化推理链。

术语表

暴力法 直接枚举解空间的方法，是理解题目和建立正确性的起点。

不变量 算法执行过程中始终成立的性质，用来证明优化解正确。

滑动窗口 用左右边界维护连续区间，并增量更新窗口状态的方法。

前缀和 把区间和查询转化为两个前缀差值的数据表示。

哈希表 通过哈希函数把 key 映射到桶，平均支持常数时间查询的数据结构。

BFS 广度优先搜索，适合无权最短路和层次扩展。

DFS 深度优先搜索，适合连通块、路径搜索和回溯。

拓扑排序 对有向无环图按依赖顺序排列节点的方法。

索引

algorithm, ii

data structure, ii

数据结构, ii

算法, ii